

SUB: - SOFT COMPUTING PRACTICAL

SR NO	PRACTICAL AIM	DATE	SIGNATURE
1	Implement the following. A. Design a simple linear network model. B. Calculate the output of neural net using both binary and bipolar sigmoidal function.		
2	Implement the following. A. Generate AND/NOT function using McCulloch-Pitts neural net. B. Generate XOR function using McCulloch-Pitts neural net.		
3	Implement the following. A. Write a program to implement Hebb 's rule. B. Write a program to implement of delta rule.		
4	Implement the following. A. Write a program for Back Propagation Algorithm. B. Write a program for error Backpropagation algorithm.		
5	Implement the following. A. Write a program for Hopfield Network. B. Write a program for Radial Basis function.		
6	Implement the following. A. Kohonen Self organizing map. B. Adaptive resonance theory.		
7	Implement the following. A. Write program for Linear separation. B. Write a program for Hopfield network model for associative memory.		
8	Implement the following. A. Membership and Identity Operators in, not in. B. Membership and Identity Operators is, is not.		
9	Implement the following. A. Find ratios using fuzzy logic. B. Solve Tipping problem using fuzzy logic.		

10	Implement the following. A. Implementation of simple genetic algorithm. B. Create two classes City and Fitness using Genetic algorithm.		
----	--	--	--

PRACTICAL 1

1(A):- Design a simple linear neural network

model. CODE: -

```

x = float(input("\n Enter the input x = ")) w =
float(input("\n Enter the weight w = ")) b
= float(input("\n Enter the bias b = "))

yin = w * x + b
print("\n Next input yin ", yin)

print("\n *****OUTPUT*****") if yin
< 0:
    output = 0
elif yin > 1:
    output = 1
else: output = yin
print("Output", output)

```

OUTPUT: -

The screenshot shows a Google Colab notebook titled 'Untitled6.ipynb'. The code in the cell is as follows:

```
print("\n *****OUTPUT*****")
if yin < 0:
    output = 0
elif yin > 1:
    output = 1
else:
    output = yin
print("Output", output)
```

The input/output section shows the following interaction:

```
Enter the input x = 0.1
Enter the weight w = 0.2
Enter the bias b = 1
Next input yin 1.02
*****OUTPUT*****
Output 1
```

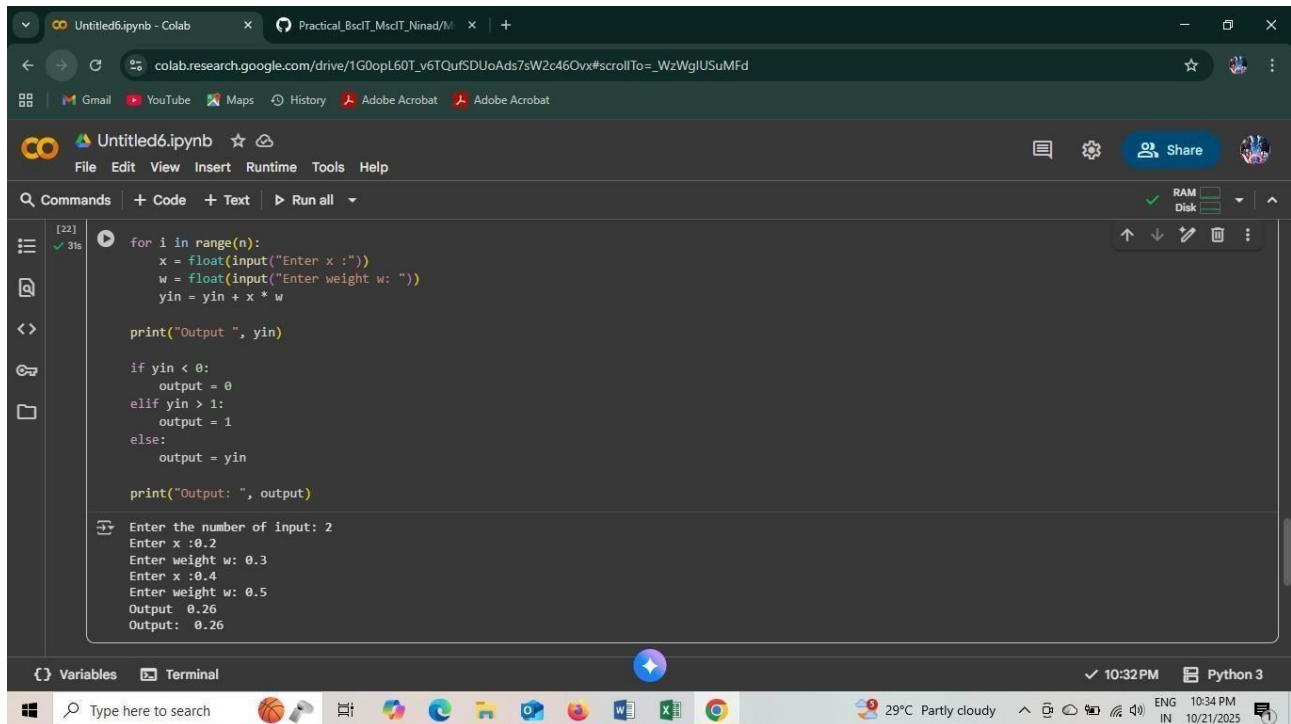
1(A)II:- Calculate the output of neural net for using data. CODE:-

```
n = int(input("Enter the number of input: ")) yin
= 0
```

```
for i in range(n): x = float(input("Enter
x :")) w = float(input("Enter weight w:
")) yin = yin + x * w print("Output ",
yin)
```

```
if yin < 0:
    output = 0
elif yin > 1:
    output = 1
else: output = yin
print("Output: ", output)
```

OUTPUT:-



```
[22] ✓ 31s
for i in range(n):
    x = float(input("Enter x :"))
    w = float(input("Enter weight w: "))
    yin = yin + x * w

    print("Output ", yin)

if yin < 0:
    output = 0
elif yin > 1:
    output = 1
else:
    output = yin

print("Output: ", output)

Enter the number of input: 2
Enter x :0.2
Enter weight w: 0.3
Enter x :0.4
Enter weight w: 0.5
Output  0.26
Output:  0.26
```

1(B):-Calculate the output of neural net using both binary and bipolar sigmoidal function.

CODE:-

```
import math
n = int(input("Enter the number of element "))
yin = 0
for i in range(0, n):
    x = float(input("Enter x :"))
    w = float(input("Enter weight w: "))
    yin = yin + x * w
```

```
b = float(input("B "))
yin = yin + b
print("yin ", yin)
```

```
binary_sigmoidal = 1 / (1 + (math.e ** (-yin)))
print("Binary Sigmoidal = ", round(binary_sigmoidal, 3))
```

```
bipolar_sigmoidal = (2 / (1 + (math.e ** (-yin)))) - 1
print("Bipolar Sigmoidal = ", round(bipolar_sigmoidal, 3))
```

OUTPUT:-

The screenshot shows a Google Colab notebook titled 'Untitled6.ipynb'. The code defines a function to calculate the output of a neural network for a given number of elements. It uses sigmoid and bipolar sigmoid functions. The output shows the results for 2 elements.

```

for i in range(0, n):
    x = float(input("Enter x :"))
    w = float(input("Enter weight w: "))
    yin = yin + x * w

    b = float(input("B "))
    yin = yin + b
    print("yin ", yin)

    binary_sigmoidal = 1 / (1 + (math.e ** (-yin)))
    print("Binary Sigmoidal = ", round(binary_sigmoidal, 3))

    bipolar_sigmoidal = (2 / (1 + (math.e ** (-yin)))) - 1
    print("Bipolar Sigmoidal = ", round(bipolar_sigmoidal, 3))

Enter the number of element 2
Enter x :0.2
Enter weight w: 0.3
Enter x :0.4
Enter weight w: 0.5
B 1
yin 1.26
Binary Sigmoidal = 0.779
Bipolar Sigmoidal = 0.558

```

PRACTICAL 2**2(A):-Generate AND/NOT function using McCulloch-Pitts neural net. CODE:-**

```

print("ANDNOT function using McCulloch-pitts\n") x1inputs
= [1, 1, 0, 0] x2inputs
= [1, 0, 1, 0]

print("Considering one weight as excitatory and other as inhibitory") w1
= [1, 1, 1, 1] w2 =
[-1, -1, -1, -1]

yin = [] print("x1", "x2",
"yin") for i in range(0, 4):
    yin.append(x1inputs[i] * w1[i] + x2inputs[i] *
w2[i]) print(x1inputs[i], " ", x2inputs[i], " ", yin[i])

theta = 2 * 1 - 1 # n *w -p
print("Threshold-theta - ", theta)
print("Applying Threshold - ",
theta) Y = [] for i in range(0, 4):
    if yin[i] >= theta: value
    = 1
    Y.append(value) else:
    value = 0

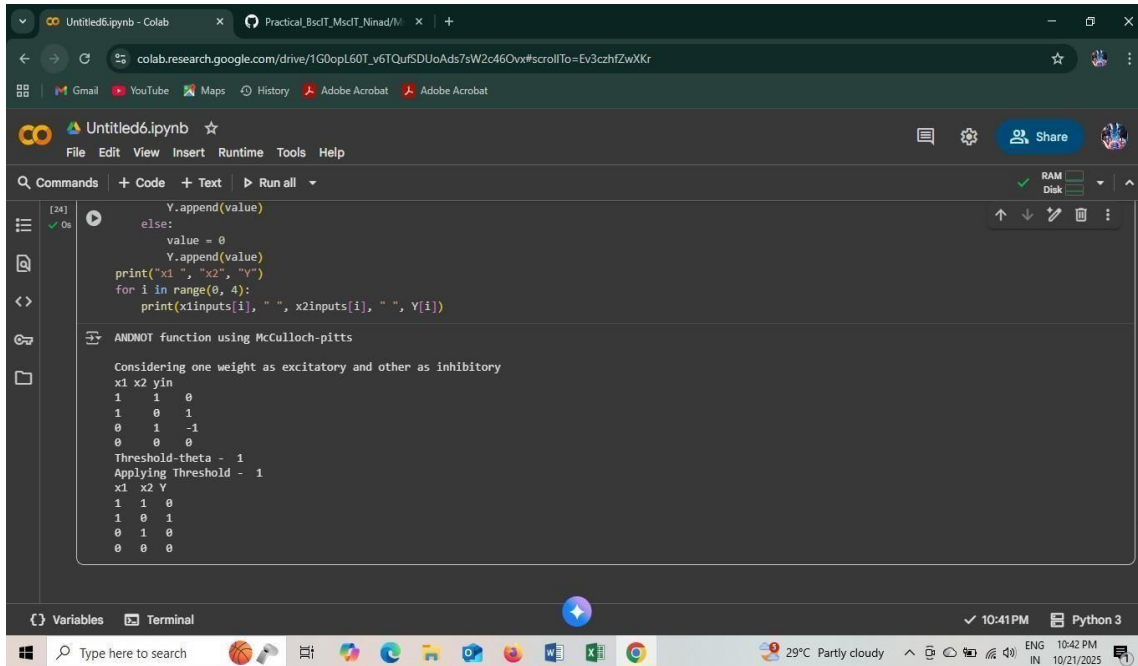
```

```

        Y.append(value)
print("x1 ", "x2",
      "Y") for i in range(0,
4):
    print(x1inputs[i], " ", x2inputs[i], " ", Y[i])

```

OUTPUT:-



The screenshot shows a Google Colab notebook titled 'Untitled6.ipynb'. The code cell contains the following Python code:

```

V.append(value)
else:
    value = 0
    V.append(value)
print("x1 ", "x2", "Y")
for i in range(0, 4):
    print(x1inputs[i], " ", x2inputs[i], " ", Y[i])

```

The output cell displays the following text:

```

ANDNOT function using McCulloch-pitts

Considering one weight as excitatory and other as inhibitory
x1 x2 yin
1 1 0
1 0 1
0 1 -1
0 0 0

Threshold-theta - 1
Applying Threshold - 1
x1 x2 Y
1 1 0
1 0 1
0 1 0
0 0 0

```

The bottom of the image shows a Windows taskbar with various application icons, a search bar, and system information including the date and time (10:42 PM, 10/21/2025).

2(B):-Generate XOR function using McCulloch-Pitts neural net.**CODE:-**

```

print("XOR function using McCulloch-pitts\n")
x1inputs = [1, 1, 0, 0] x2inputs
= [1, 0, 1, 0]

print("Calculating z1 = x1w11 + x2w21") print("Considering one
weights as excitatory and other as inhibitory") w11 = [1, 1, 1, 1] w21
= [-1, -1, -1, -1]

print("x1", "x2", "z1")
z1 = [] for i in range(0,
4):
    z1.append(x1inputs[i] * w11[i] + x2inputs[i] *
w21[i]) print(x1inputs[i], " ", x2inputs[i], " ", z1[i])

# Z2 print("Calculating z2 = x1w12 + x2w22") print("Considering
one weights as inhibitory and other as excitatory") w12 = [-1, -1, -1, -
1] w22 = [1, 1, 1, 1]

print("x1", "x2", "z2")
z2 = [] for i in range(0,
4):
    z2.append(x1inputs[i] * w12[i] + x2inputs[i] *
w22[i]) print(x1inputs[i], " ", x2inputs[i], " ", z2[i])

print("Applying threshold for x1 and x2") for
i in range(0, 4): if z1[i] >= 1:
    z1[i] = 1
else:
    z1[i] = 0
    if z2[i] >= 1:
        z2[i] = 1
    else:
        z2[i] = 0
print("z1", "z2") for i
in range(0, 4):
    print(z1[i], " ", z2[i])

print("x1 ", "x2",
"Yin") yin = [] v1 = 1
v2 = 1 for i in range(0,
4): yin.append(z1[i] *

```

```

v1 + z2[i] * v2)
print(x1inputs[i], " ",
x2inputs[i], " ", yin[i])

print("Applying Threshold=1 for yin") print("x1",
"x2", "yin") y =
[] for i in
range(0, 4):
    if yin[i] >= 1:
        y.append(1)
    else:
        y.append(0)
for i in range(0, 4):
    print(x1inputs[i], " ", x2inputs[i], " ", y[i]) print("END")

```

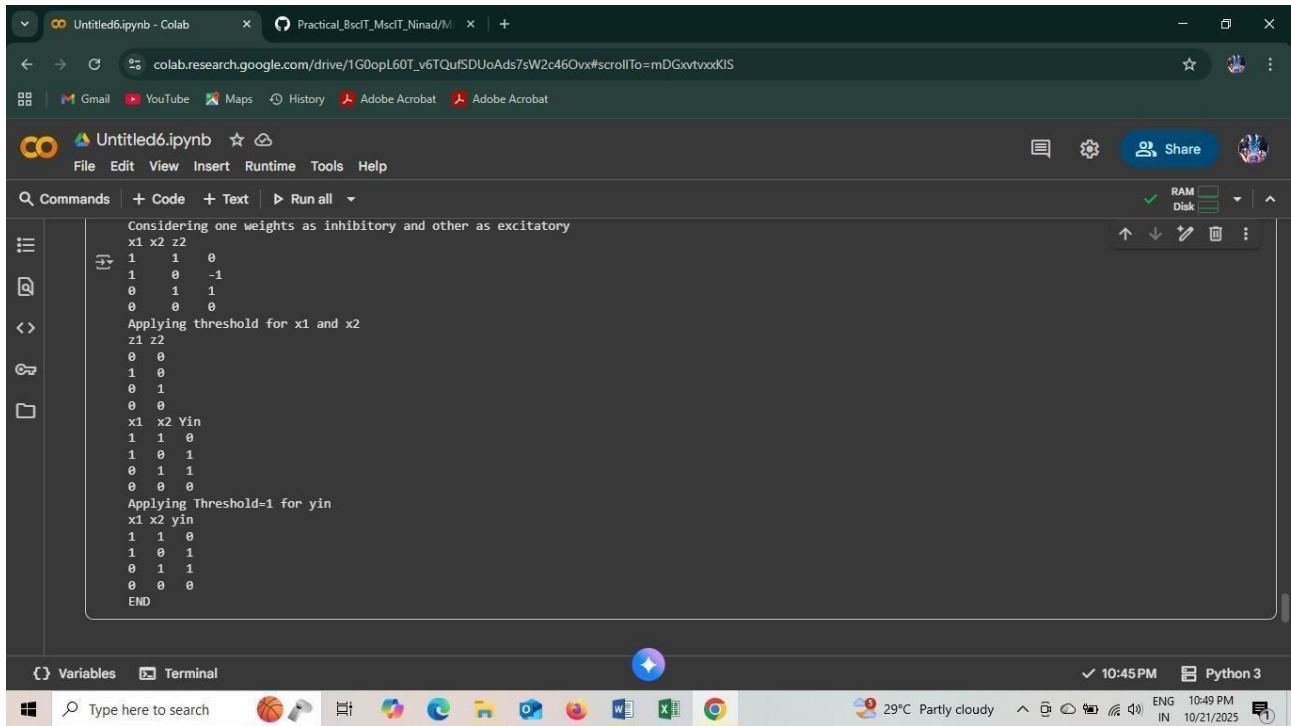
OUTPUT:-

```

XOR function using McCulloch-pitts
Calculating z1 = x1w11 + x2w21
Considering one weights as excitatory and other as inhibitory
x1 x2 z1
1 1 0
1 0 1
0 1 -1
0 0 0
Calculating z2 = x1w12 + x2w22
Considering one weights as inhibitory and other as excitatory
x1 x2 z2
1 1 0
1 0 -1
0 1 1
0 0 0
Applying threshold for x1 and x2
z1 z2
0 0
1 0
0 1
0 0
x1 x2 Yin
1 1 0
1 0 1
0 1 1
0 1 1

```


SUB: - SOFT COMPUTING PRACTICAL



```
Untitled6.ipynb - Colab
colab.research.google.com/drive/1G0opL60T_v6TQufSDUoAds7sW2c46Ovx#scrollTo=mDGxvtvxxkIS
Gmail YouTube Maps History Adobe Acrobat Adobe Acrobat
Untitled6.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
RAM Disk
Considering one weights as inhibitory and other as excitatory
x1 x2 z2
1 1 0
1 0 -1
0 1 1
0 0 0
Applying threshold for x1 and x2
z1 z2
0 0
1 0
0 1
0 0
x1 x2 Yin
1 1 0
1 0 1
0 1 1
0 0 0
Applying Threshold=1 for yin
x1 x2 yin
1 1 0
1 0 1
0 1 1
0 0 0
END
Variables Terminal
10:45 PM Python 3
29°C Partly cloudy 10:49 PM 10/21/2025
```

PRACTICAL 3

3(A):- Write a program to implement Hebb's rule.

CODE:-

```
import numpy as np

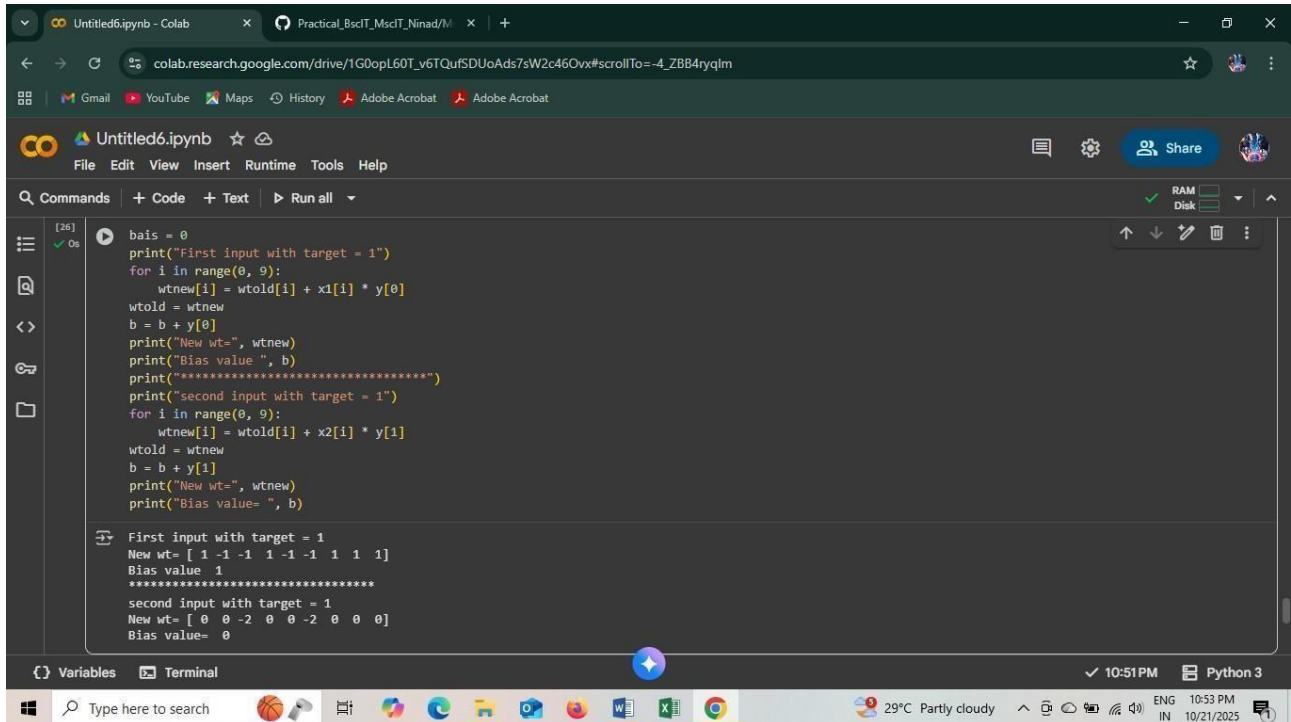
x1 = np.array([1, -1, -1, 1, -1, -1, 1, 1, 1]) x2
= np.array([1, -1, 1, 1, -1, 1, 1, 1, 1]) b
= 0

y = np.array([1, -1])
wtold = np.zeros((9,))
wtnew = np.zeros((9,))

wtnew = wtnew.astype(int) wtold
= wtold.astype(int)

bais = 0 print("First input with target
= 1") for i in range(0, 9):
    wtnew[i] = wtold[i] + x1[i] * y[0]
wtold = wtnew b = b +
y[0] print("New wt=",
wtnew)
print("Bias          value          ", b)
print("*****")
) print("second input with target = 1") for i in range(0,
9):
    wtnew[i] = wtold[i] + x2[i] * y[1]
wtold = wtnew b = b + y[1]
print("New wt=",
wtnew) print("Bias
value= ", b)
```

SUB: - SOFT COMPUTING PRACTICAL



The screenshot displays a Google Colab notebook titled 'Untitled6.ipynb'. The notebook contains a Python script for a simple neural network training process. The script initializes a bias value to 0 and iterates through two different input sets, each with a target value of 1. For each input, it calculates new weights and updates the bias. The output of the script is displayed in the cell's output area.

```
bais = 0
print("First input with target = 1")
for i in range(0, 9):
    wtnew[i] = wtold[i] + x1[i] * y[0]
    wtold = wtnew
    b = b + y[0]
print("New wt=", wtnew)
print("Bias value ", b)
print("*****")
print("second input with target = 1")
for i in range(0, 9):
    wtnew[i] = wtold[i] + x2[i] * y[1]
    wtold = wtnew
    b = b + y[1]
print("New wt=", wtnew)
print("Bias value= ", b)
```

First input with target = 1
New wt= [1 -1 -1 1 -1 -1 1 1 1]
Bias value 1

second input with target = 1
New wt= [0 0 -2 0 0 -2 0 0 0]
Bias value= 0

OUTPUT:-**3(B):-Write a program to implement of delta rule.****CODE:-**

```

print("\b Aim: - Write a program to implement delta rule")
import numpy as np import time

np.set_printoptions(precision=2) x =
np.zeros((3,)) weights      =
np.zeros((3,)) desired      =
np.zeros((3,)) actual = np.zeros((3,))
for i in range(0,
3): x[i] =
float(input("Intial inputs: "))
for i in range(0, 3): weights[i]
= float(input("Intial weights: "))
for i in range(0, 3):
    desired[i] = float(input("Desired output: "))
a = float(input("Enter learning rate: "))
actual = x * weights print("Actual ",
actual) print("Desired ", desired) while
True: if np.array_equal(desired, actual):
break else:
    for i in range(0, 3): weights[i] = weights[i] + a *
(desired[i] - actual[i])
    actual = x * weights
print("Weights      ", weights)
print("Actual ", actual)
print("Desired ", desired) print("*"
* 30) print("Final output")
print("Corrected weights", weights)
print("Actual", actual)
print("Desired ", desired)

```

SUB: - SOFT COMPUTING PRACTICAL

The screenshot shows a Google Colab notebook titled 'Untitled6.ipynb'. The browser address bar shows the URL: `colab.research.google.com/drive/1G0opL60T_v6TQufSDUoAds7sW2c46Ovx#scrollTo=OL03y1GB1OXr`. The notebook interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help), a toolbar with 'Share' and 'RAM Disk' indicators, and a left sidebar with icons for file management. The code cell contains the following Python code:

```
[27] print("Corrected weights", weights)
      print("Actual", actual)
      print("Desired ", desired)
```

Below the code, the notebook displays the output of the program:

```
b) Aim: - Write a program to implement delta rule
Initial inputs: 1
Initial inputs: 1
Initial inputs: 1
Initial weights: 1
Initial weights: 1
Initial weights: 1
Desired output: 2
Desired output: 3
Desired output: 4
Enter learning rate: 1
Actual [1. 1. 1.]
Desired [2. 3. 4.]
Weights [2. 3. 4.]
Actual [2. 3. 4.]
Desired [2. 3. 4.]
*****
Final output
Corrected weights [2. 3. 4.]
Actual [2. 3. 4.]
Desired [2. 3. 4.]
```

The bottom of the image shows a Windows taskbar with various application icons, a search bar, and system status information including temperature (29°C), weather (Partly cloudy), and date/time (11:05 PM, 10/21/2025).

OUTPUT:-**PRACTICAL 4****4(A):-Write a program for Back Propagation Algorithm.****CODE:-**

```

import numpy as np

X = np.array([2, 9], [1, 5], [3, 6]), dtype=float)
Y = np.array([92], [86], [89]), dtype=float)
X = X / np.amax(X, axis=0)
Y = Y / 100

class NN(object):
    def init_(self):
        self.inputsize = 2
        self.outputsize = 1
        self.hiddensize = 3
        self.W1 = np.random.randn(self.inputsize, self.hiddensize)
        self.W2 = np.random.randn(self.hiddensize, self.outputsize)

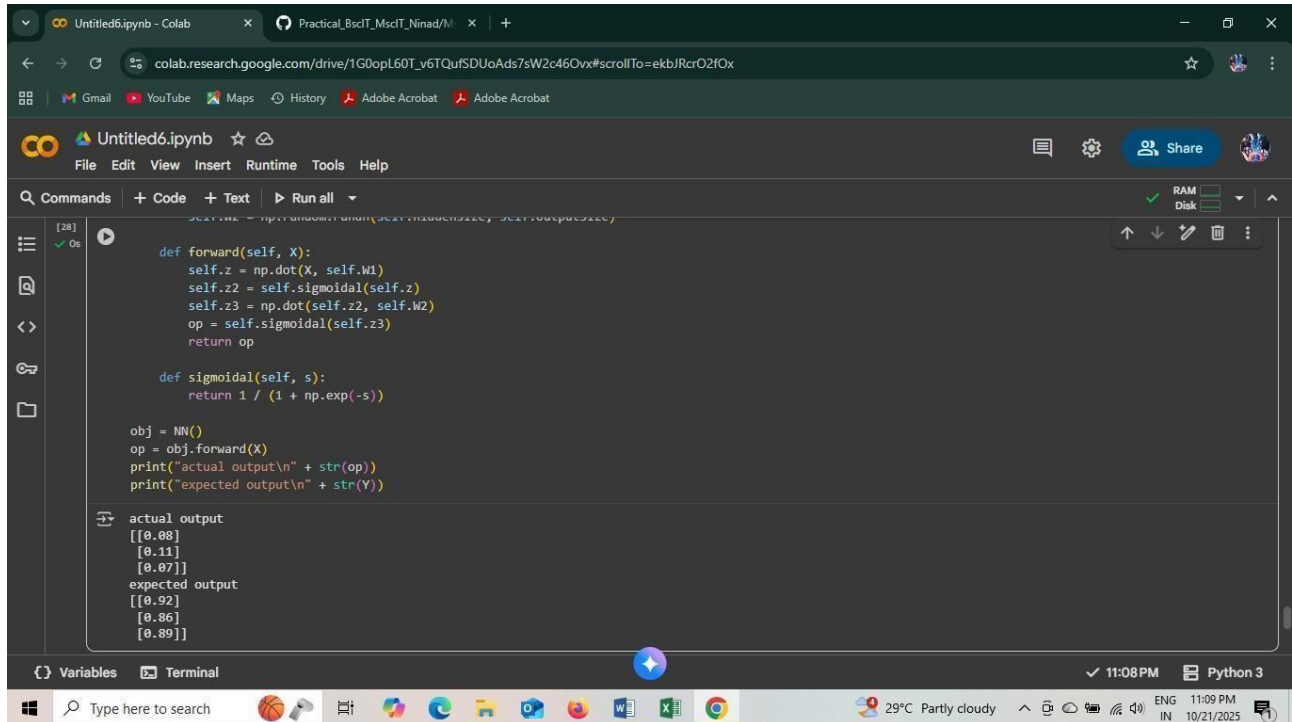
    def forward(self, X):
        self.z = np.dot(X, self.W1)
        self.z2 = self.sigmoidal(self.z)
        self.z3 = np.dot(self.z2, self.W2)
        op = self.sigmoidal(self.z3)
        return op

    def sigmoidal(self, s):
        return 1 / (1 + np.exp(-s))

obj = NN()
op = obj.forward(X)
print("actual output\n" + str(op))
print("expected output\n" + str(Y))

```

SUB: - SOFT COMPUTING PRACTICAL



The screenshot shows a Google Colab notebook titled "Untitled6.ipynb". The notebook contains a Python script for a simple neural network. The script defines two functions: `forward` and `sigmoidal`. The `forward` function takes an input `X` and a neural network object `self` and returns the output `op`. The `sigmoidal` function takes a value `s` and returns the sigmoid value `1 / (1 + np.exp(-s))`. The script then creates an instance of the neural network `obj = NN()`, calls the `forward` function with input `X`, and prints the actual output `op`. The output is displayed as a list of values: `[0.08]`, `[0.11]`, and `[0.07]`. The expected output is also displayed as a list of values: `[0.92]`, `[0.86]`, and `[0.89]`.

```
[28] ✓ 0s

def forward(self, X):
    self.z = np.dot(X, self.W1)
    self.z2 = self.sigmoidal(self.z)
    self.z3 = np.dot(self.z2, self.W2)
    op = self.sigmoidal(self.z3)
    return op

def sigmoidal(self, s):
    return 1 / (1 + np.exp(-s))

obj = NN()
op = obj.forward(X)
print("actual output\n" + str(op))
print("expected output\n" + str(Y))

actual output
[0.08]
[0.11]
[0.07]
expected output
[0.92]
[0.86]
[0.89]
```

OUTPUT:-**4(B):-Write a program for error Backpropagation algorithm.****CODE:-**

```

import numpy as np

X = np.array([[2, 9], [1, 5], [3, 6]], dtype=float)
Y = np.array([[92], [86], [89]], dtype=float)

X = X / np.amax(X, axis=0)
Y = Y / 100

class NN(object):
    def init_(self):
        self.inputsize = 2 self.outputsize
        = 1 self.hiddensize = 3
        self.W1 = np.random.randn(self.inputsize, self.hiddensize) self.W2
        = np.random.randn(self.hiddensize, self.outputsize)

    def forward(self, X): self.z =
        np.dot(X, self.W1) self.z2 =
        self.sigmoidal(self.z) self.z3 =
        np.dot(self.z2, self.W2) op =
        self.sigmoidal(self.z3) return op

    def sigmoidal(self, s):
        return 1 / (1 + np.exp(-s))

    def sigmoidalprime(self, s): return s
        * (1 - s)

    def backward(self, X, Y, o):
        self.o_error = Y - o self.o_delta = self.o_error *
        self.sigmoidalprime(o) self.z2_error =
        self.o_delta.dot(self.W2.T) self.z2_delta = self.z2_error *
        self.sigmoidalprime(self.z2) self.W1 = self.W1 +
        X.T.dot(self.z2_delta) self.W2 = self.W2 +
        self.z2.T.dot(self.o_delta)

    def train(self, X, Y): o =
        self.forward(X)

```


SUB: - SOFT COMPUTING PRACTICAL

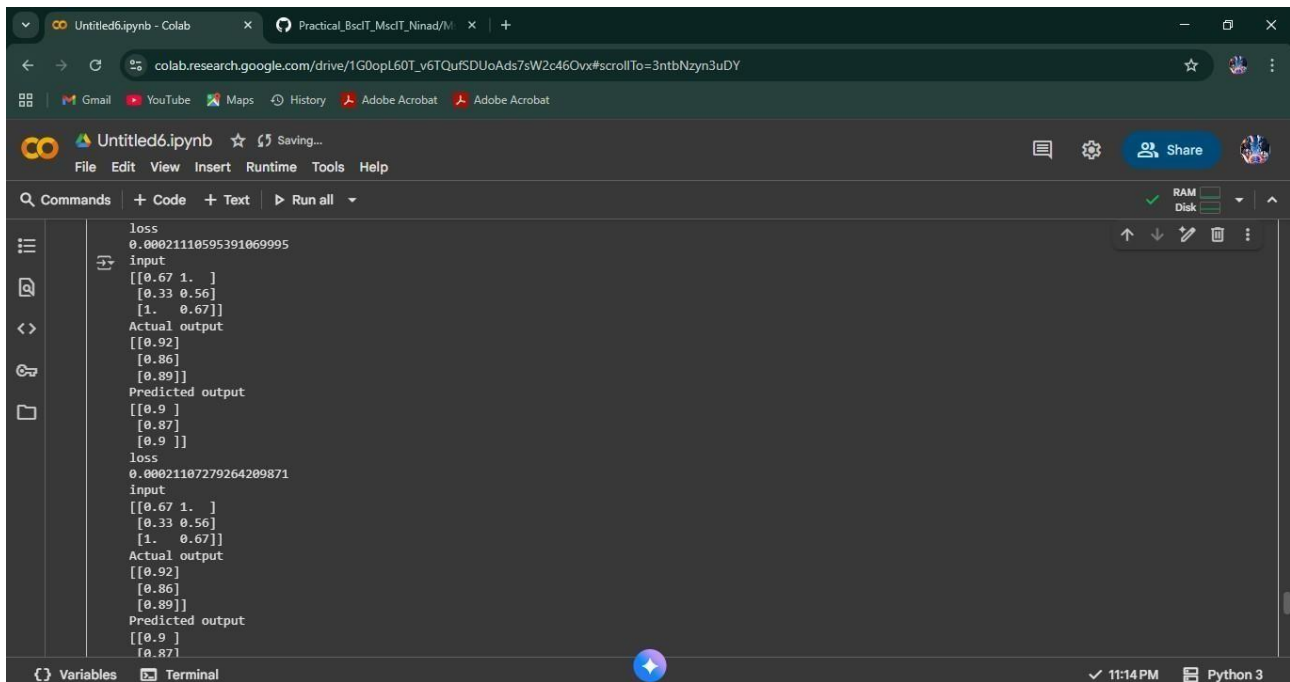
```
self.backward(X, Y,  
o)
```

```
obj = NN() for i in range(2000):
```

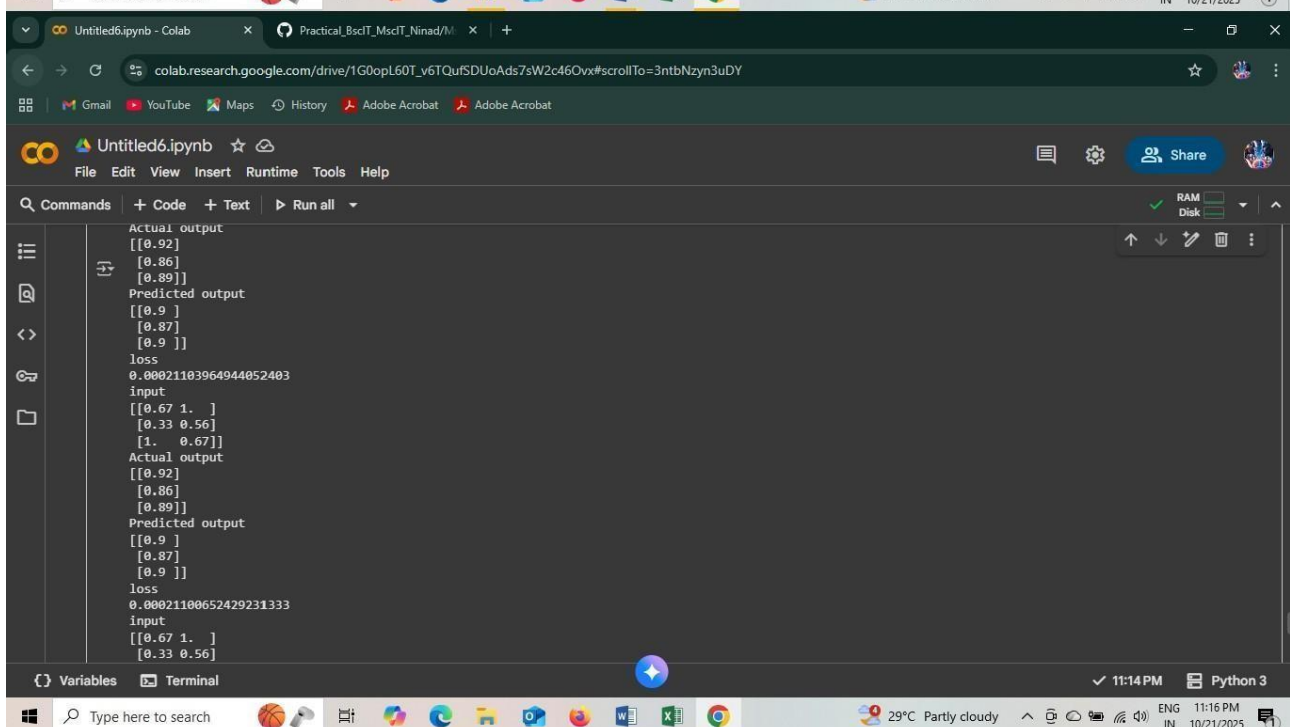
SUB: - SOFT COMPUTING PRACTICAL

```
print("input\n" + str(X)) print("Actual  
output\n" + str(Y)) print("Predicted  
output\n" + str(obj.forward(X)))  
print("loss\n" +  
str(np.mean(np.square(Y -  
obj.forward(X)))) obj.train(X, Y)
```

OUTPUT:-

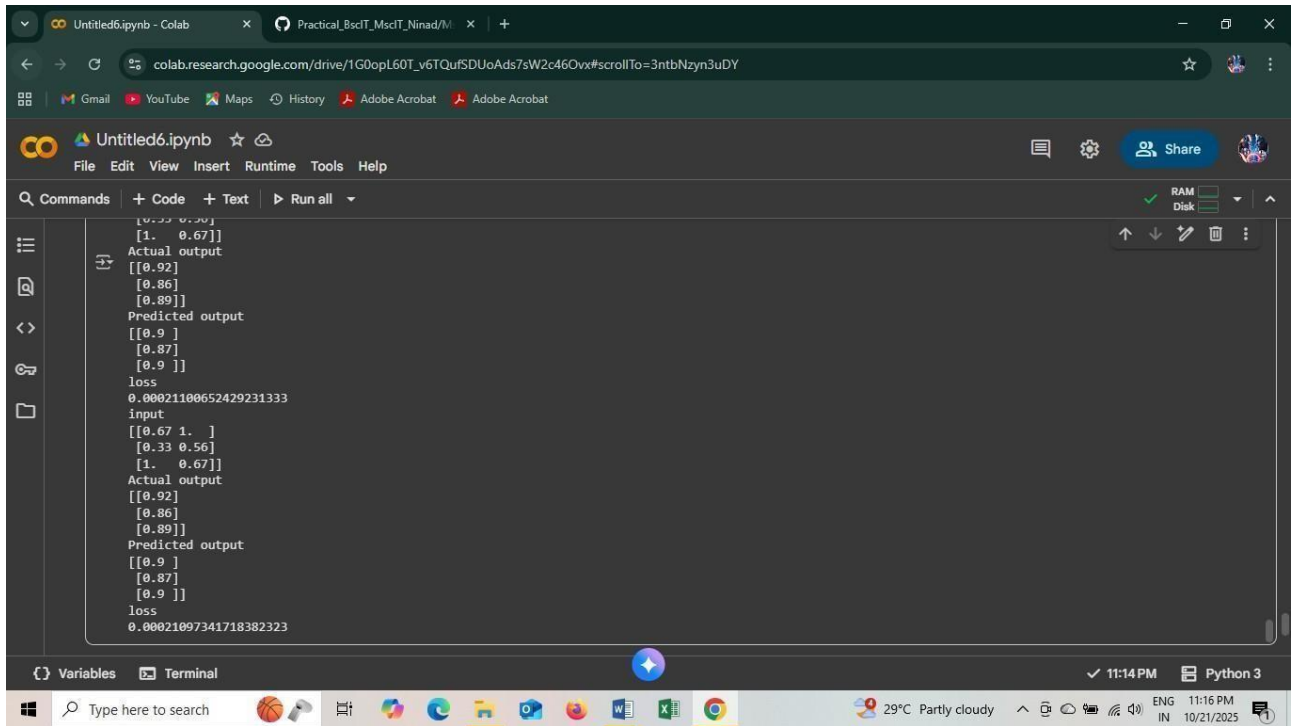


```
loss  
0.00021110595391069995  
input  
[[0.67 1. ]  
 [0.33 0.56]  
 [1.  0.67]]  
Actual output  
[[0.92]  
 [0.86]  
 [0.89]]  
Predicted output  
[[0.9 ]  
 [0.87]  
 [0.9 ]]  
loss  
0.00021107279264209871  
input  
[[0.67 1. ]  
 [0.33 0.56]  
 [1.  0.67]]  
Actual output  
[[0.92]  
 [0.86]  
 [0.89]]  
Predicted output  
[[0.9 ]  
 [0.87]  
 [0.87]]
```



```
Actual output  
[[0.92]  
 [0.86]  
 [0.89]]  
Predicted output  
[[0.9 ]  
 [0.87]  
 [0.9 ]]  
loss  
0.00021103964944052403  
input  
[[0.67 1. ]  
 [0.33 0.56]  
 [1.  0.67]]  
Actual output  
[[0.92]  
 [0.86]  
 [0.89]]  
Predicted output  
[[0.9 ]  
 [0.87]  
 [0.9 ]]  
loss  
0.00021100652429231333  
input  
[[0.67 1. ]  
 [0.33 0.56]
```

SUB: - SOFT COMPUTING PRACTICAL



The screenshot shows a Google Colab notebook titled 'Untitled6.ipynb'. The notebook contains a Python script that outputs the following results:

```
[0.33 0.56]
[1. 0.67]]
Actual output
[[0.92]
 [0.86]
 [0.89]]
Predicted output
[[0.9 ]
 [0.87]
 [0.9 ]]
loss
0.00021100652429231333
input
[[0.67 1. ]
 [0.33 0.56]
 [1. 0.67]]
Actual output
[[0.92]
 [0.86]
 [0.89]]
Predicted output
[[0.9 ]
 [0.87]
 [0.9 ]]
loss
0.00021097341718382323
```

The bottom of the notebook interface shows the 'Variables' and 'Terminal' tabs, and the system tray at the bottom indicates the time is 11:14 PM on 10/21/2025, with a temperature of 29°C and 'Partly cloudy' weather.

PRACTICAL 5

5(A):-Write a program for Hopfield Network.

CODE:-

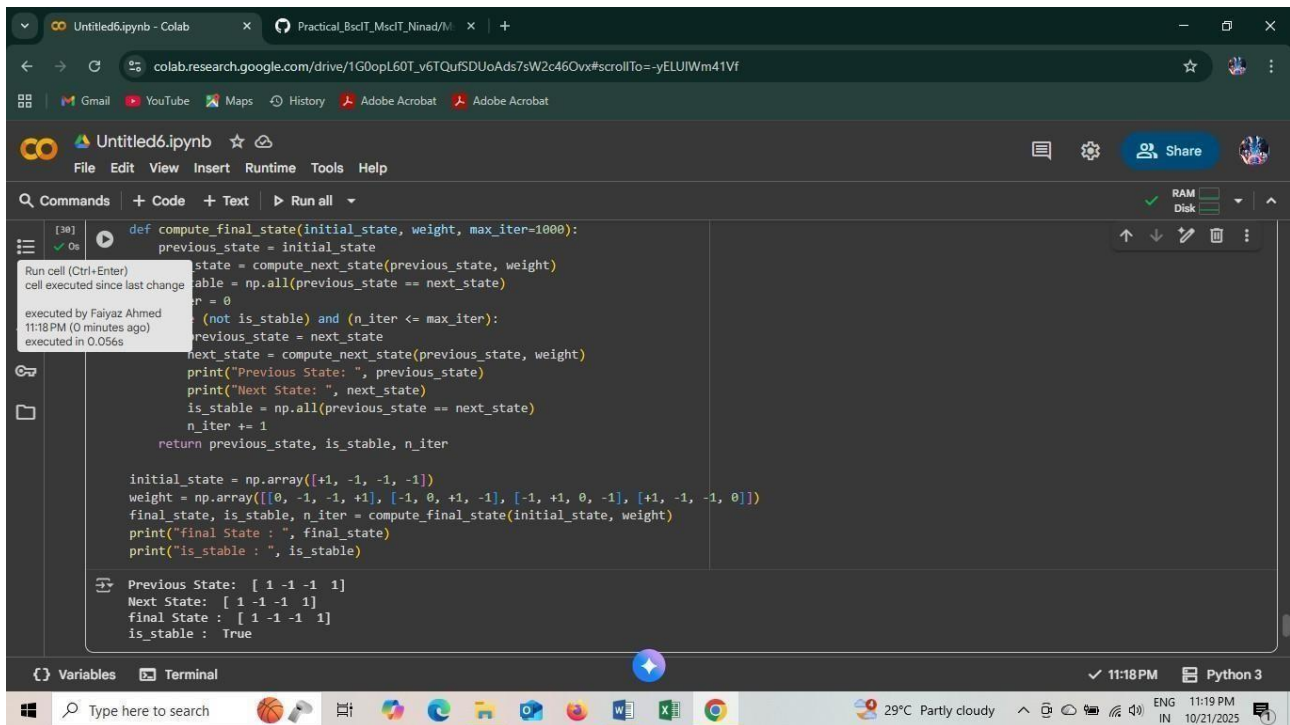
```
import numpy as np
def compute_next_state(state, weight):
    # @ is a shorthand for 'np.matmul()'
    # the numpy.where() function return thr indices of #
    # element in an input array where condition is satisfied
    next_state = np.where(weight @ state >= 0, +1, -1)
    return next_state

def compute_final_state(initial_state, weight, max_iter=1000):
    previous_state = initial_state
    next_state = compute_next_state(previous_state, weight)
    is_stable = np.all(previous_state == next_state)
    n_iter = 0
    while (not is_stable) and (n_iter <= max_iter):
        previous_state = next_state
        next_state = compute_next_state(previous_state, weight)
        print("Previous State: ", previous_state)
        print("Next State: ", next_state)
        is_stable = np.all(previous_state == next_state)
        n_iter += 1
    return previous_state, is_stable, n_iter

initial_state = np.array([+1, -1, -1, -1])
weight = np.array([[0, -1, -1, +1], [-1, 0, +1, -1], [-1, +1, 0, -1], [+1, -1, -1, 0]])
final_state, is_stable, n_iter = compute_final_state(initial_state, weight)
print("final State : ", final_state)
print("is_stable : ", is_stable)
```

SUB: - SOFT COMPUTING PRACTICAL

OUTPUT:-



The screenshot displays a Google Colab notebook interface. The top bar shows the notebook name 'Untitled6.ipynb' and the Google Drive URL. The left sidebar contains icons for file management and a 'Run' button. The main area shows a Python function `compute_final_state` with the following code:

```
def compute_final_state(initial_state, weight, max_iter=1000):  
    previous_state = initial_state  
    state = compute_next_state(previous_state, weight)  
    stable = np.all(previous_state == next_state)  
    n_iter = 0  
    while (not is_stable) and (n_iter <= max_iter):  
        previous_state = next_state  
        next_state = compute_next_state(previous_state, weight)  
        print("Previous State: ", previous_state)  
        print("Next State: ", next_state)  
        is_stable = np.all(previous_state == next_state)  
        n_iter += 1  
    return previous_state, is_stable, n_iter  
  
initial_state = np.array([+1, -1, -1, -1])  
weight = np.array([[0, -1, -1, +1], [-1, 0, +1, -1], [+1, -1, -1, 0]])  
final_state, is_stable, n_iter = compute_final_state(initial_state, weight)  
print("final State : ", final_state)  
print("is_stable : ", is_stable)
```

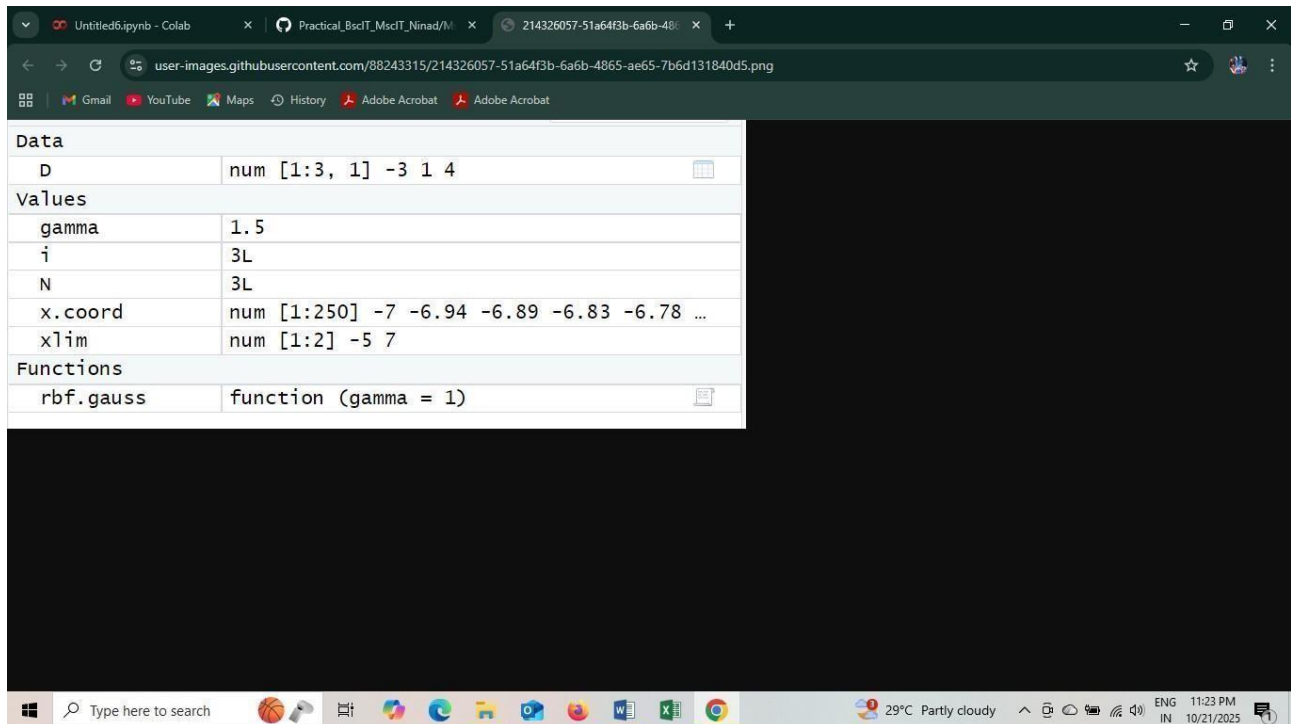
The output of the function is displayed below the code:

```
Previous State: [ 1 -1 -1  1]  
Next State: [ 1 -1 -1  1]  
final State : [ 1 -1 -1  1]  
is_stable : True
```

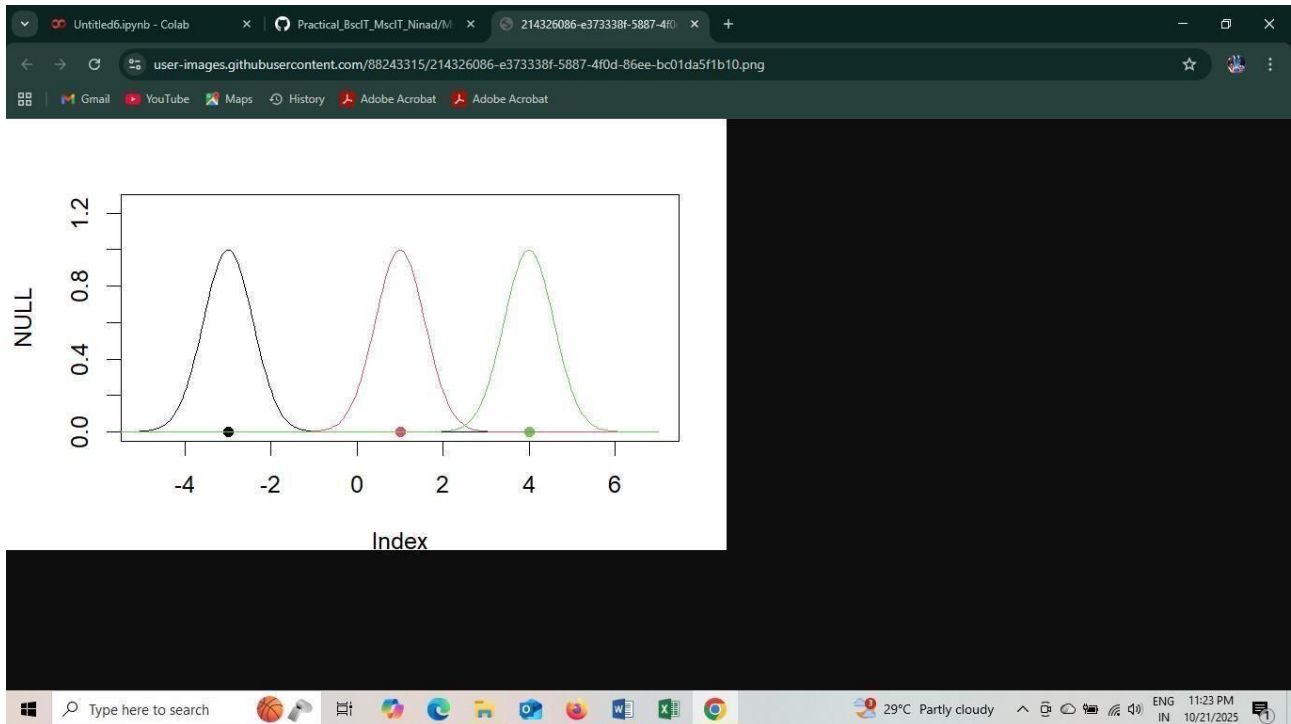
The bottom status bar shows the system time as 11:18 PM, the Python version as Python 3, and the system temperature as 29°C Partly cloudy.

5(B):-Write a program for Radial Basis function.**CODE:-**

```
# R language
D <- matrix(c(-3, 1, 4), ncol = 1) # 3 datapoints N
<- length(D) rbf.gauss <-
function(gamma = 1.0) { function(x)
{
  exp(-gamma * norm(as.matrix(x), "F")^2) } }
xlim <- c(-5, 7) print(N) print(xlim) plot(NULL,
xlim = xlim, ylim = c(0, 1.25), type = "n") points(D,
rep(0, length(D)), col = 1:N, pch = 19) x.coord <-
seq(-7, 7, length = 250)
gamma <- 1.5 for (i in 1:N) { points(x.coord, lapply(x.coord - D[i, ],
rbf.gauss(gamma)), type = "l", col = i) }
```

OUTPUT:-

SUB: - SOFT COMPUTING PRACTICAL



PRACTICAL 6

6(A):-Kohonen Self organizing map.

CODE:-

```
from minisom import MiniSom import  
numpy as np import matplotlib.pyplot  
as plt
```

```
colors = [  
    [0.0, 0.0, 0.0],  
    [0.0, 0.0, 1.0],  
    [0.0, 0.0, 0.5],  
    [0.125, 0.529, 1.0],  
    [0.33, 0.4, 0.67],  
    [0.6, 0.5, 1.0],  
    [0.0, 1.0, 0.0],  
    [1.0, 0.0, 0.0],  
    [0.0, 1.0, 1.0],  
    [1.0, 0.0, 1.0],  
    [1.0, 1.0, 0.0],  
    [1.0, 1.0, 1.0],  
    [0.33, 0.33, 0.33], [0.5,  
    0.5, 0.5],  
    [0.66, 0.66, 0.66],  
]
```

```
color_names = [  
    "black",  
    "blue",  
    "darkblue",  
    "skyblue",  
    "greyblue",  
    "lilac",  
    "green",  
    "red",  
    "cyan",  
    "violet",  
    "yellow",  
    "white",  
    "darkgrey",  
    "mediumgrey",  
    "lightgrey",  
]
```


SUB: - SOFT COMPUTING PRACTICAL

```
]

som = MiniSom(30, 30, 3, sigma=3.0, learning_rate=2.5, neighborhood_function="gaussian")

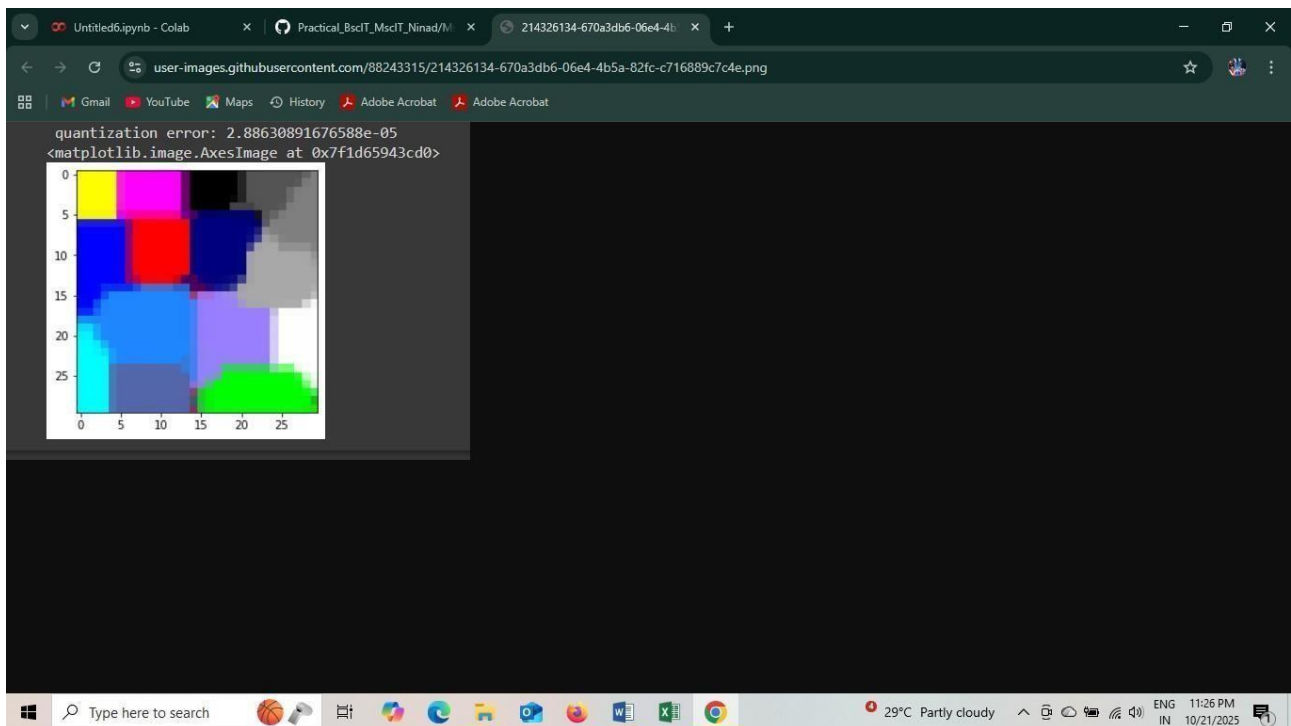
plt.imshow(abs(som.get_weights()), interpolation="none") som = MiniSom(30, 30, 3, sigma=8.0,

learning_rate=0.5, neighborhood_function="bubble")

som.train_random(colors, 500, verbose=True)

plt.imshow(abs(som.get_weights()), interpolation="none")
```

OUTPUT:-



6(B):-Adaptive resonance theory.

CODE:-

```
from neurodynex.hopfield_network import network, pattern_tools, plot_tools import matplotlib.pyplot
as plt

pattern_size = 5
# create an instance of the class HopfieldNetwork hopfield_net =
network.HopfieldNetwork(nr_neurons=pattern_size ** 2)
```

SUB: - SOFT COMPUTING PRACTICAL

```
# instantiate a pattern factory factory =
pattern_tools.PatternFactory(pattern_size, pattern_size)

# create a checkerboard pattern and add it to the pattern list
checkerboard = factory.create_checkerboard() pattern_list
= [checkerboard]

# add random patterns to the list
pattern_list.extend( factory.create_random_pattern_list(nr_patterns=3,
on_probability=0.5)
)
plot_tools.plot_pattern_list(pattern_list)

# how similar are the random patterns and the checkerboard? Check the overlaps
overlap_matrix = pattern_tools.compute_overlap_matrix(pattern_list)
plot_tools.plot_overlap_matrix(overlap_matrix)

# let the hopfield network "learn" the patterns. Note: they are not stored #
explicitly but only network weights are updated !
hopfield_net.store_patterns(pattern_list)

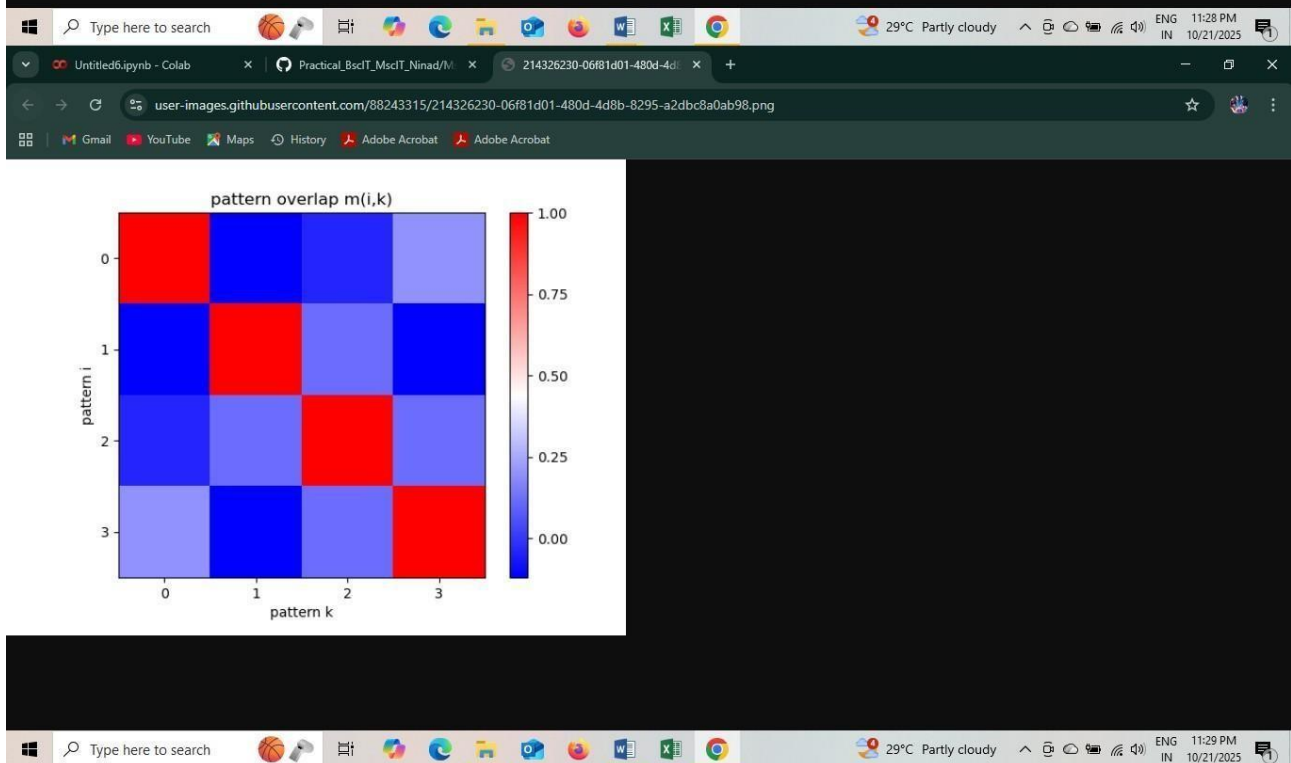
# create a noisy version of a pattern and use that to initialize the network
noisy_init_state = pattern_tools.flip_n(checkerboard, nr_of_flips=4)
hopfield_net.set_state_from_pattern(noisy_init_state)

# from this initial state, let the network dynamics evolve. states
= hopfield_net.run_with_monitoring(nr_steps=4)

# each network state is a vector. reshape it to the same shape used to create the patterns.
states_as_patterns = factory.reshape_patterns(states)

# plot the states of the network plot_tools.plot_state_sequence_and_overlap(
states_as_patterns, pattern_list, reference_idx=0, subtitle="Network dynamics"
)
```

OUTPUT:-



PRACTICAL 7

7(A):-Write a program for Linear separation.

CODE:-

```
import numpy as np import matplotlib.pyplot
as plt

class Perceptron:
    def init_(self, learning_rate=0.01, n_iterations=1000):
        self.learning_rate = learning_rate self.n_iterations =
        n_iterations self.weights = None self.bias = None

    def fit(self, X, y):
        n_samples, n_features = X.shape

        # Initialize weights and bias self.weights
        = np.zeros(n_features) self.bias
        = 0

        # Convert labels to -1 and 1 for easier calculation y_
        = np.where(y == 0, -1, 1)

        for _ in range(self.n_iterations):
            for idx, x_i in enumerate(X): linear_output =
                np.dot(x_i, self.weights) + self.bias prediction =
                np.sign(linear_output)

            # Update weights and bias if misclassified if prediction
            != y_[idx]:
                self.weights += self.learning_rate * y_[idx] * x_i self.bias
                += self.learning_rate * y_[idx]

    def predict(self, X):
        linear_output = np.dot(X, self.weights) + self.bias return
        np.where(np.sign(linear_output) == -1, 0, 1)

# Generate synthetic linearly separable data np.random.seed(42)
X = np.random.rand(100, 2) * 10 # 100 samples, 2 features y = np.where(X[:,
0] + X[:, 1] > 10, 1, 0) # Create a linear separation

# Create and train the Perceptron perceptron =
    Perceptron(learning_rate=0.01, n_iterations=100)
perceptron.fit(X, y)
```

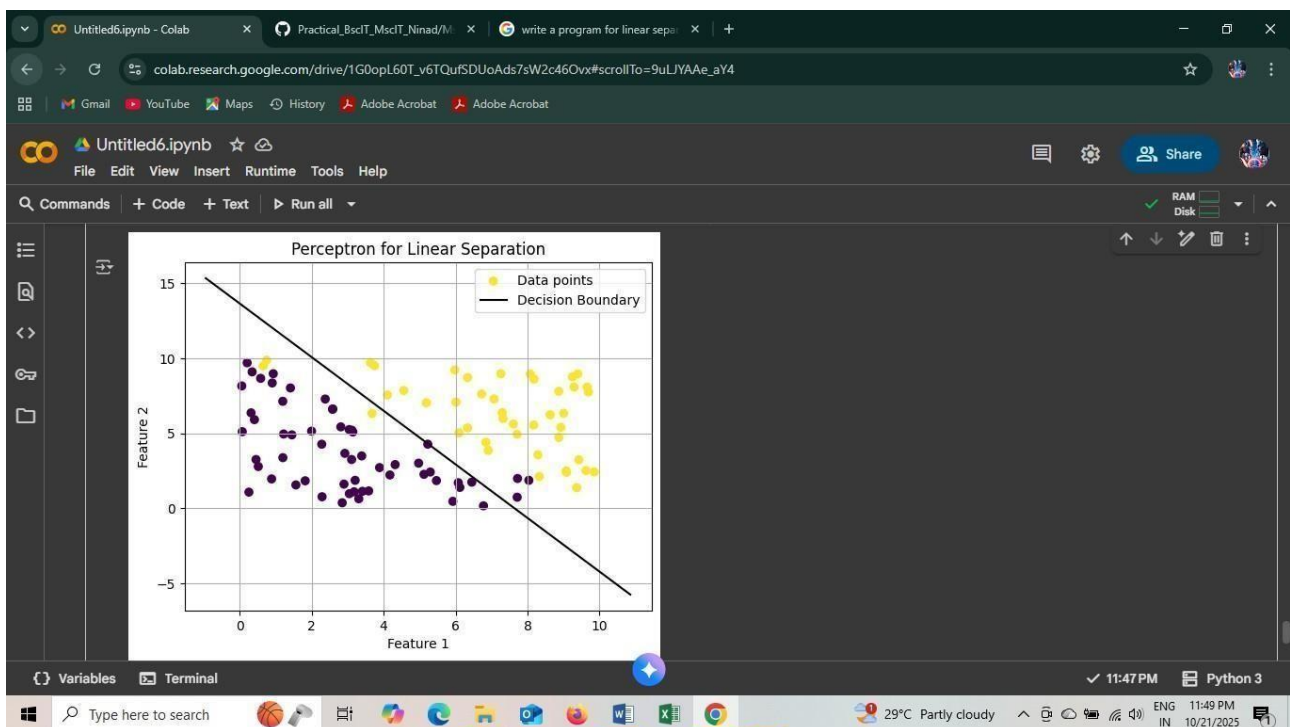
```
# Make predictions predictions =
perceptron.predict(X)

# Plot the data and decision boundary plt.scatter(X[:, 0], X[:, 1],
c=y, cmap='viridis', label='Data points')

# Plot the decision boundary (hyperplane) x_min,
x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() +
1

xx = np.linspace(x_min, x_max, 100) yy = (-perceptron.weights[0] * xx -
perceptron.bias) / perceptron.weights[1]

plt.plot(xx, yy, 'k-', label='Decision
Boundary') plt.xlabel('Feature 1')
plt.ylabel('Feature 2') plt.title('Perceptron for
Linear Separation') plt.legend() plt.grid(True)
plt.show()
```

OUTPUT:-

7(B) Write a program for Hopfield network model for associative memory.

CODE:-

```
import numpy as np
```

```

class HopfieldNetwork:
    def init_(self, num_neurons):
        self.num_neurons = num_neurons
        self.weights = np.zeros((num_neurons, num_neurons))

    def train(self, patterns): """
        Trains the Hopfield network using Hebbian learning rule.
        Patterns should be a list of 1D numpy arrays, where each element is -1 or 1.
        """
        for pattern in patterns:
            # Outer product of the pattern with itself
            # Add to weights, ensuring no self-connections (diagonal elements are 0)
            self.weights += np.outer(pattern, pattern) - np.fill_diagonal(self.weights, 0)
            # No self-connections

    def recall(self, input_pattern, max_iterations=100):
        """
        Recalls a stored pattern from a given input pattern.
        """
        current_state = np.copy(input_pattern)
        for _ in range(max_iterations):
            prev_state = np.copy(current_state)
            # Asynchronous update: randomly select a neuron to update for _ in range(self.num_neurons):
            neuron_idx = np.random.randint(self.num_neurons)
            net_input = np.dot(self.weights[neuron_idx], current_state)
            current_state[neuron_idx] = 1 if net_input >= 0 else -1

            # Check for convergence if np.array_equal(current_state, prev_state):
            break
        return current_state

# Example Usage:
if __name__ == "__main__":
    # Define patterns to be stored (must be bipolar: -1 or 1)
    pattern1 = np.array([1, 1, 1, -1, -1, -1, -1, 1, 1])
    pattern2 = np.array([-1, -1, -1, 1, 1, 1, 1, -1, -1])
    pattern3 = np.array([1, -1, 1, -1, 1, -1, 1, -1, 1])
    patterns = [pattern1, pattern2, pattern3]
    num_neurons = len(pattern1)
    hopfield_net = HopfieldNetwork(num_neurons)

    print("Training the Hopfield Network...")
    hopfield_net.train(patterns)
    print("Training complete.")

```

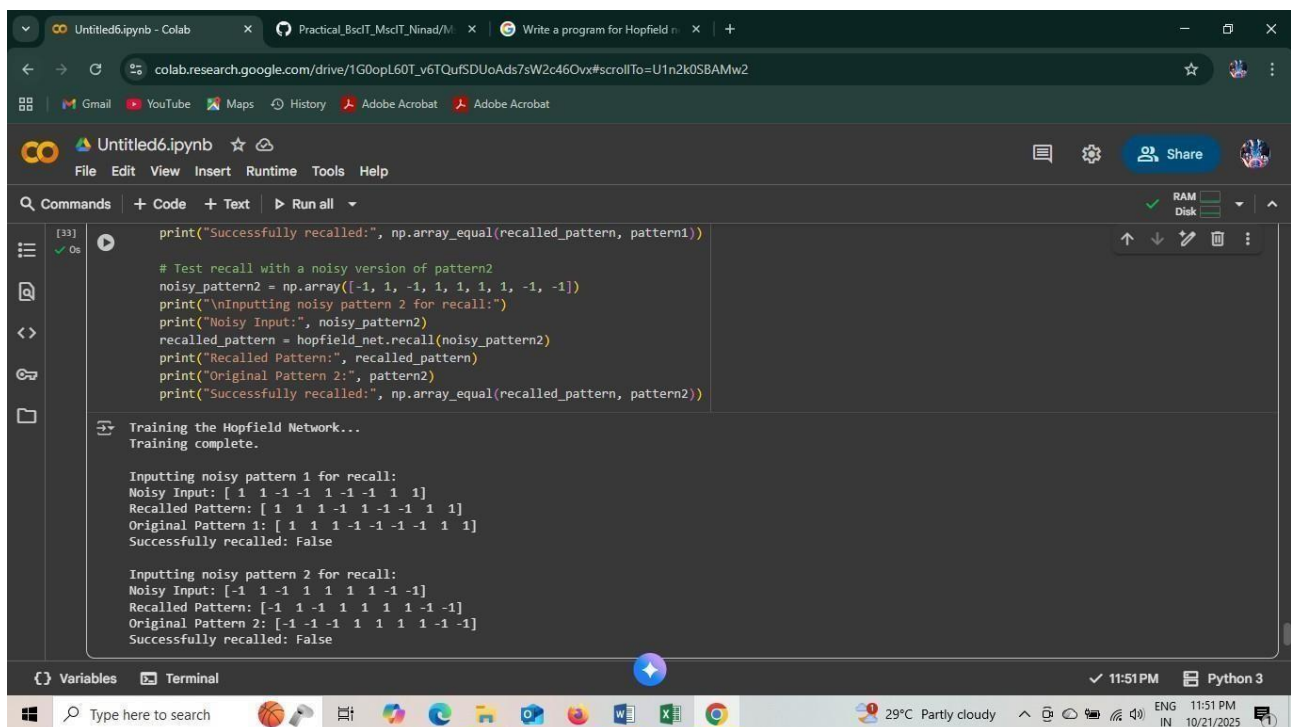
SUB: - SOFT COMPUTING PRACTICAL

```
# Test recall with a noisy version of pattern1 noisy_pattern1 = np.array([1, 1, -1, -1, 1, -1, -1, 1, 1]) # Pattern1 with some flipped bits
```

```
print("\nInputting noisy pattern 1 for recall:") print("Noisy Input:", noisy_pattern1)
recalled_pattern = hopfield_net.recall(noisy_pattern1)
print("Recalled Pattern:", recalled_pattern)
print("Original Pattern 1:", pattern1) print("Successfully recalled:", np.array_equal(recalled_pattern, pattern1))
```

```
# Test recall with a noisy version of pattern2
noisy_pattern2 = np.array([-1, 1, -1, 1, 1, 1, 1, -1, -1]) print("\nInputting noisy pattern 2 for recall:")
print("Noisy Input:", noisy_pattern2)
recalled_pattern = hopfield_net.recall(noisy_pattern2)
print("Recalled Pattern:", recalled_pattern)
print("Original Pattern 2:", pattern2)
print("Successfully recalled:", np.array_equal(recalled_pattern, pattern2))
```

OUTPUT:-



```
print("Successfully recalled:", np.array_equal(recalled_pattern, pattern1))

# Test recall with a noisy version of pattern2
noisy_pattern2 = np.array([-1, 1, -1, 1, 1, 1, 1, -1, -1])
print("\nInputting noisy pattern 2 for recall:")
print("Noisy Input:", noisy_pattern2)
recalled_pattern = hopfield_net.recall(noisy_pattern2)
print("Recalled Pattern:", recalled_pattern)
print("Original Pattern 2:", pattern2)
print("Successfully recalled:", np.array_equal(recalled_pattern, pattern2))
```

Training the Hopfield Network...
Training complete.

Inputting noisy pattern 1 for recall:
Noisy Input: [1 1 -1 -1 1 -1 1 1]
Recalled Pattern: [1 1 1 -1 -1 -1 1 1]
Original Pattern 1: [1 1 1 -1 -1 -1 1 1]
Successfully recalled: False

Inputting noisy pattern 2 for recall:
Noisy Input: [-1 1 -1 1 1 1 1 -1 -1]
Recalled Pattern: [-1 1 -1 1 1 1 1 -1 -1]
Original Pattern 2: [-1 -1 1 1 1 1 1 -1 -1]
Successfully recalled: False

PRACTICAL 8

8(A):-Membership and Identity Operators in.

CODE:-

```
list1 = [] print("Enter 5
numbers") for i in
range(0, 5): v =
int(input())
list1.append(v) list2 = []
print("Enter 5 numbers")
for i in range(0, 5): v =
int(input())
list2.append(v) flag = 0
for i in list1:
    if i in list2:
        flag = 1
if flag == 1: print("The Lists
Overlap") else: print("The
Lists do Not overlap")
```

OUTPUT:-

The screenshot shows a Google Colab notebook interface. The code cell contains the following Python code:

```
[37] for i in range(0, 5):
      v = int(input())
      list2.append(v)
      flag = 0
      for i in list1:
          if i in list2:
              flag = 1
      if flag == 1:
          print("The Lists Overlap")
      else:
          print("The Lists do Not overlap")
```

The output cell shows the following text:

```
Enter 5 numbers
1
2
3
4
5
Enter 5 numbers
4
5
6
7
8
The Lists Overlap
```

The interface also shows a file explorer on the left, a terminal at the bottom, and a system tray at the very bottom indicating the time is 11:58 PM on 10/22/2025.

8(A)II:-Membership and Identity Operators not in.**CODE:-**

```
list1 = [] c = int(input("Enter the number of elements that you want to insert in
List 1:")) for i in range(0, c): ele = int(input("Enter the element :"))
list1.append(ele) a = int(input("enter the number that you want to find in List
1:")) if a not in list1:
    print("The list does not contain ", a)
else: print("The list contains",
    a)
```

OUTPUT:-

The screenshot shows a Google Colab notebook titled 'Untitled6.ipynb'. The code cell contains the following Python code:

```
# 7A-ii. Implement membership and identity operators *not in*.

list1 = []
c = int(input("Enter the number of elements that you want to insert in List 1:"))
for i in range(0, c):
    ele = int(input("Enter the element :"))
    list1.append(ele)
a = int(input("enter the number that you want to find in List 1:"))
if a not in list1:
    print("The list does not contain ", a)
else:
    print("The list contains", a)
```

The output of the code is as follows:

```
Enter the number of elements that you want to insert in List 1:4
Enter the element :1
Enter the element :2
Enter the element :3
Enter the element :4
enter the number that you want to find in List 1:2
The list contains 2
```

8(A)I:-Membership and Identity Operators is.**CODE:-**

```
details = [] name = input("Enter your name : ")
details.append(name) age = float(input("Enter your exact age :
")) details.append(age) roll_no = int(input("Enter your roll no :
")) details.append(roll_no) for i in details:
    print(i)
    print("Int = ", type(i) is int)
    print("Float = ", type(i) is float)
    print("String = ", type(i) is str)
    print()
```

OUTPUT:-

```

print("Not Int = ", type(i) is not int)
print("Not Float = ", type(i) is not float)
print("Not String = ", type(i) is not str)
print()

Enter your name : ninad
Enter your exact age : 21
Enter your roll no : 1012
ninad
Int = False
Float = False
String = True

21.0
Int = False
Float = True
String = False

1012
Int = True
Float = False
String = False

```

8(B)II:-Membership and Identity Operators is not.**CODE:-**

```

details = [] name = input("Enter your name : ")
details.append(name) age = float(input("Enter your exact age : "))
details.append(age) roll_no = int(input("Enter your roll no : "))
details.append(roll_no) print() for i in
details:
print(i)
    print("Not Int = ", type(i) is not int)
    print("Not Float = ", type(i) is not float)
    print("Not String = ", type(i) is not str)
    print()

```

OUTPUT:-

```

[40] ✓ 11s
for i in details:
    print(i)
    print("Not Int = ", type(i) is not int)
    print("Not Float = ", type(i) is not float)
    print("Not String = ", type(i) is not str)
    print()

Enter your name : ninad
Enter your exact age : 21
Enter your roll no : 1012

ninad
Not Int = True
Not Float = True
Not String = False

21.0
Not Int = True
Not Float = False
Not String = True

1012
Not Int = False
Not Float = True
Not String = True

```

PRACTICAL 9**9(A):-Find ratios using fuzzy logic.****CODE:-**

```

from fuzzywuzzy import fuzz
from fuzzywuzzy import process

```

```

s1 = "My name is ABC" s2 = "I am ABC" print("FuzzyWuzzy
Ratio:", fuzz.ratio(s1, s2)) print("FuzzyWuzzy    PartialRatio:  ",
    fuzz.partial_ratio(s1, s2)) print("FuzzyWuzzy
TokenSortRatio: ", fuzz.token_sort_ratio(s1, s2))
print("FuzzyWuzzy TokenSetRatio: ", fuzz.token_set_ratio(s1, s2))
print("FuzzyWuzzy WRatio: ", fuzz.WRatio(s1, s2), "\n\n")

```

```

# for process library, query = "fuzzys for fuzzys" choices = ["fuzzy for
fuzzy", "fuzzy fuzzy", "g. for fuzzys"] print("List of ratios: ")
print(process.extract(query, choices), "\n") print("Best among the above
list: ", process.extractOne(query, choices))

```

OUTPUT:-

```

8A. Find ratios using fuzzy logic.

FuzzyWuzzy Ratio: 64

FuzzyWuzzy PartialRatio: 62

FuzzyWuzzy TokenSortRatio: 55

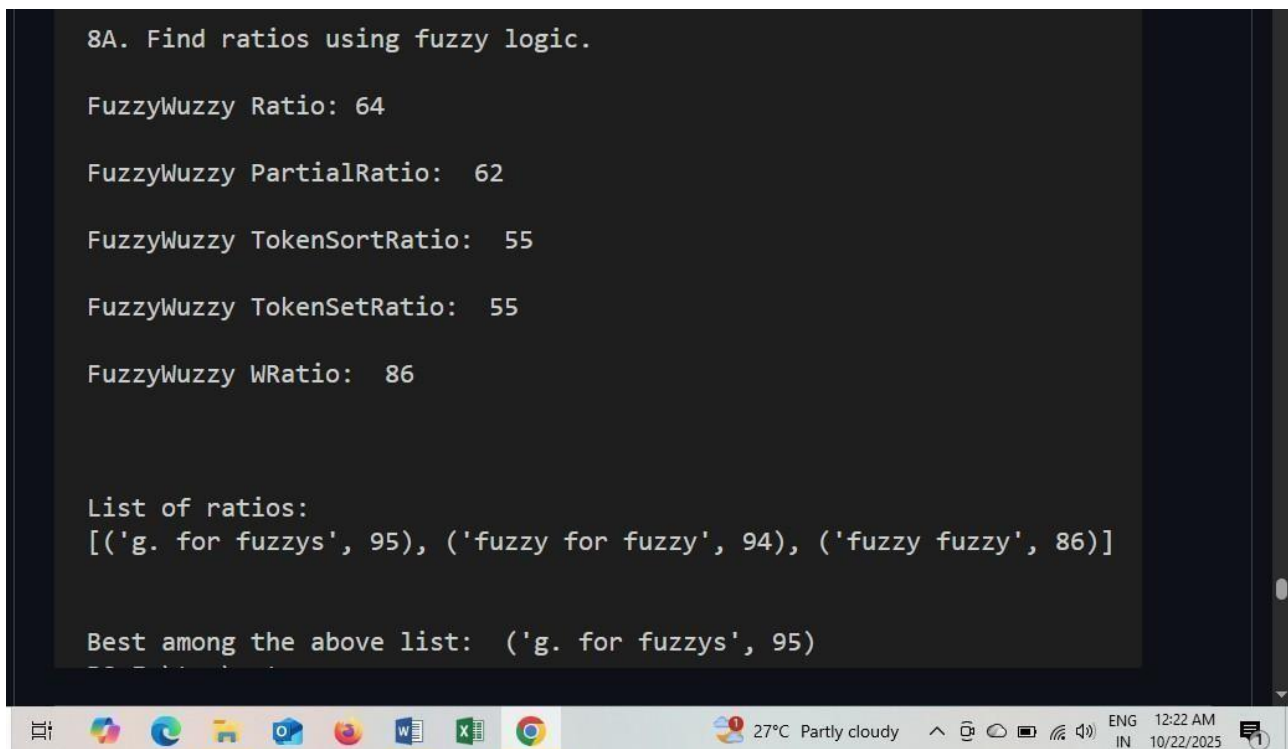
FuzzyWuzzy TokenSetRatio: 55

FuzzyWuzzy WRatio: 86

List of ratios:
[('g. for fuzzys', 95), ('fuzzy for fuzzy', 94), ('fuzzy fuzzy', 86)]

Best among the above list: ('g. for fuzzys', 95)

```


9(B):-Solve Tipping problem using fuzzy logic.**CODE:-**

```

# google.colab
!pip install fuzzywuzzy
!pip install -U scikit-fuzzy

import skfuzzy as fuzz from skfuzzy import control as
ctrl import numpy as np quality =
ctrl.Antecedent(np.arange(0, 11, 1), 'quality') service
= ctrl.Antecedent(np.arange(0, 11, 1), 'service') tip =
ctrl.Consequent(np.arange(0, 26, 1), 'tip')

quality.automf(3)
service.automf(3)

tip['low'] = fuzz.trimf(tip.universe, [0, 0, 13]) tip['medium']
= fuzz.trimf(tip.universe, [0, 13, 25]) tip['high']
= fuzz.trimf(tip.universe, [13, 25, 25])

quality['average'].view(
) service.view()
tip.view()

```

SUB: - SOFT COMPUTING PRACTICAL

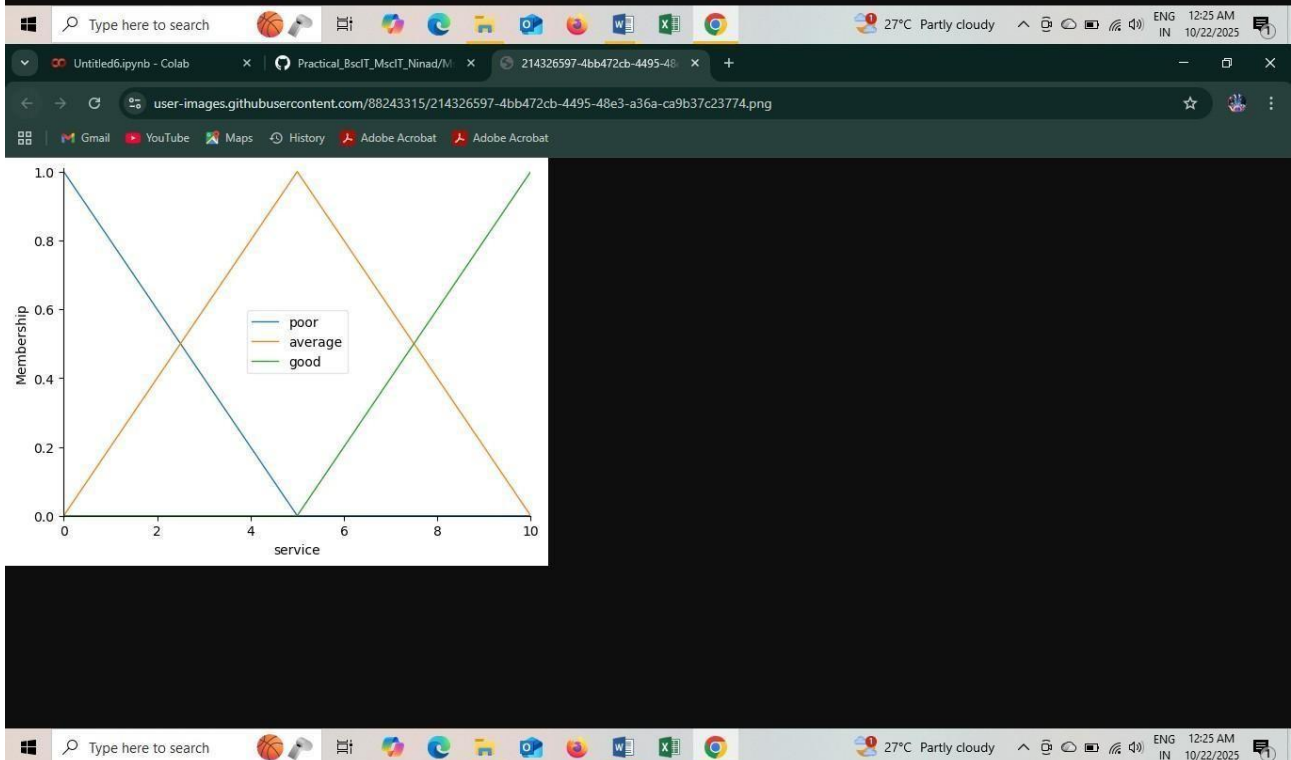
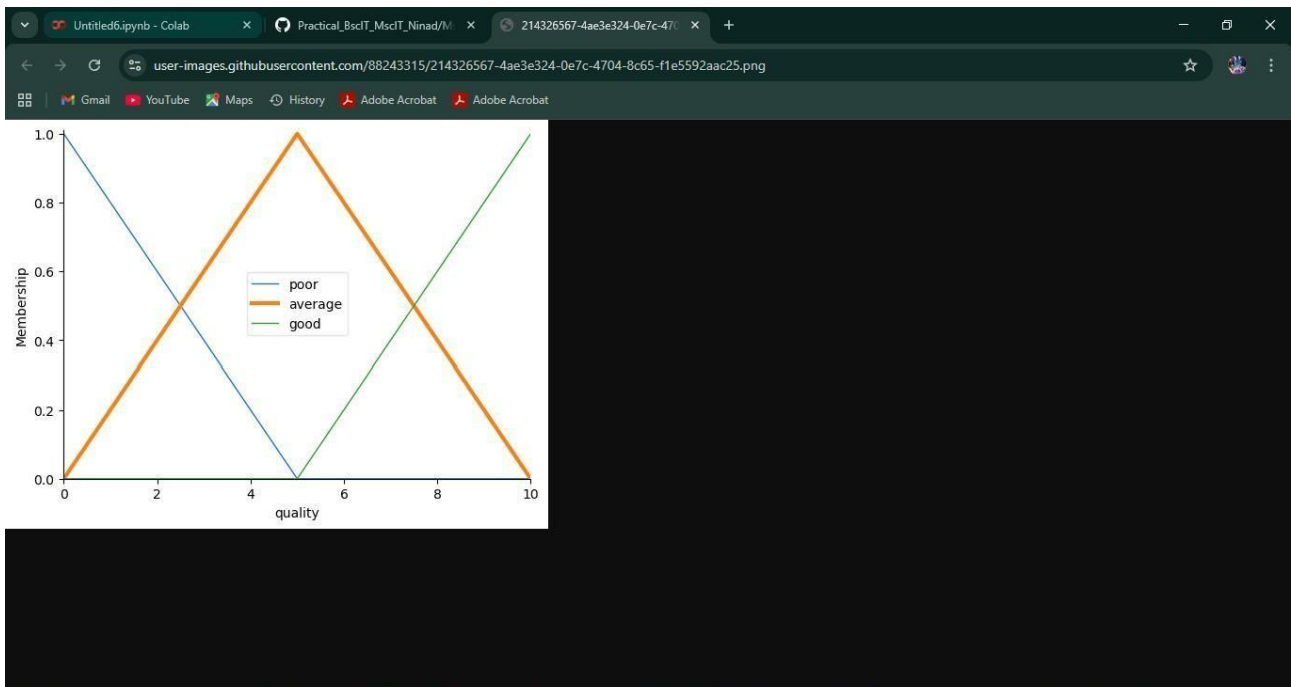
```
rule1 = ctrl.Rule(quality['poor'] | service['poor'], tip['low'])
rule2 = ctrl.Rule(service['average'], tip['medium']) rule3 =
ctrl.Rule(service['good'] | quality['good'],
tip['high']) rule1.view()

tipping_ctrl = ctrl.ControlSystem([rule1, rule2, rule3]) tipping
= ctrl.ControlSystemSimulation(tipping_ctrl)

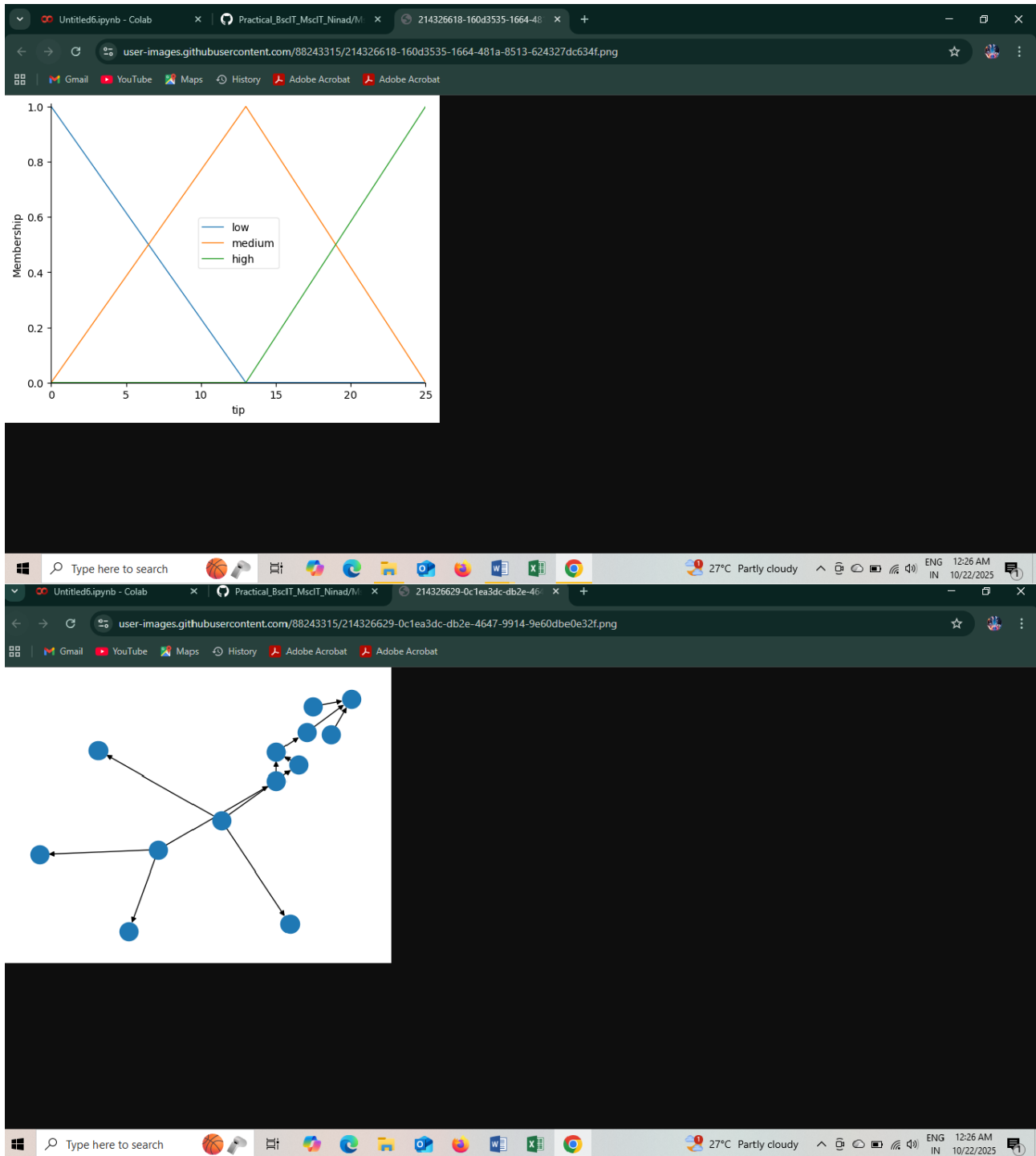
tipping.input['quality'] = 6.5 tipping.input['service']
= 9.8

tipping.compute() print
(tipping.output['tip'])
tip.view(sim=tipping)
```

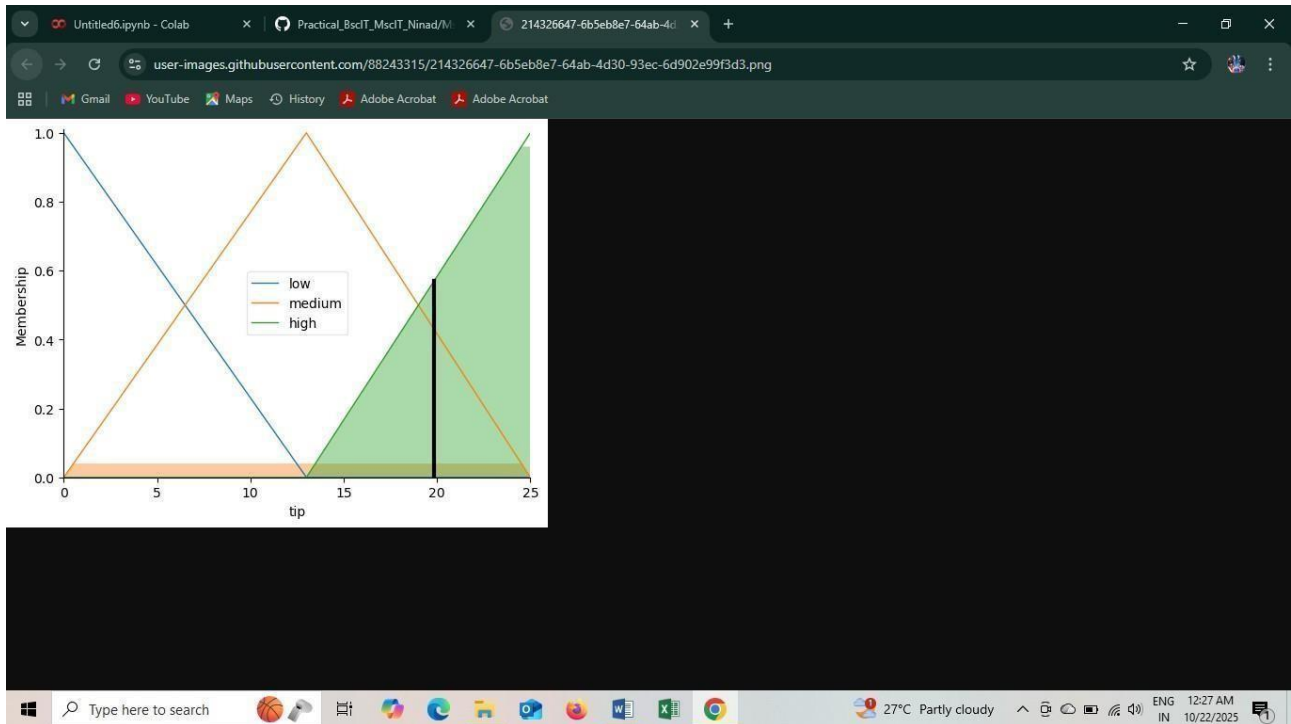
OUTPUT:-



SUB: - SOFT COMPUTING PRACTICAL



SUB: - SOFT COMPUTING PRACTICAL



PRACTICAL 10

10(A):-Implementation of Simple genetic algorithm.

CODE:-

```
import random

# Define problem-specific parameters
POPULATION_SIZE = 100
GENOME_LENGTH = 10 # Length of the binary string representing a solution
MUTATION_RATE = 0.01
CROSSOVER_RATE = 0.8
NUM_GENERATIONS = 50

# 1. Fitness Function (Example: Maximize the number of '1's in a binary string)
def calculate_fitness(individual): return sum(individual)

# 2. Initialization def create_initial_population(size,
genome_length): population = [] for _ in range(size):
    individual = [random.randint(0, 1) for _ in range(genome_length)]
    population.append(individual)
return population

# 3. Selection (Example: Roulette Wheel Selection) def
    select_parents(population, fitness_scores):
total_fitness = sum(fitness_scores) selection_probabilities = [f / total_fitness for f in fitness_scores]
    parents_indices = random.choices(range(len(population)), weights=selection_probabilities, k=2)
    return population[parents_indices[0]], population[parents_indices[1]]

# 4. Crossover (Example: Single-Point Crossover) def crossover(parent1,
parent2):
    if random.random() < CROSSOVER_RATE:
        crossover_point = random.randint(1, GENOME_LENGTH - 1)
        offspring1 = parent1[:crossover_point] + parent2[crossover_point:]
        offspring2 = parent2[:crossover_point] + parent1[crossover_point:]
        return offspring1, offspring2
    else:
        return parent1,
        parent2

# 5. Mutation def
mutate(individual):
    for i in range(GENOME_LENGTH):
        if random.random() < MUTATION_RATE:
            individual[i] = 1 - individual[i] # Flip the bit
```

return individual

```
# Main Genetic Algorithm Loop def
run_genetic_algorithm():
    population = create_initial_population(POPULATION_SIZE, GENOME_LENGTH)

    for generation in range(NUM_GENERATIONS):
        fitness_scores = [calculate_fitness(individual) for individual in population]

        new_population = [] for _ in range(POPULATION_SIZE // 2): #
            Create pairs of offspring parent1, parent2 =
            select_parents(population, fitness_scores) offspring1,
            offspring2 = crossover(parent1, parent2)
            new_population.append(mutate(offspring1))
            new_population.append(mutate(offspring2))

        population = new_population

        best_individual = max(population, key=calculate_fitness)
        best_fitness = calculate_fitness(best_individual)
        print(f"Generation {generation+1}: Best fitness = {best_fitness}, Best individual =
        {best_individual}") return best_individual

# Execute the algorithm best_solution = run_genetic_algorithm() print(f"\nFinal best solution
found: {best_solution} with fitness: {calculate_fitness(best_solution)}")
```

OUTPUT:-

```
print(f"\nFinal best solution found: {best_solution} with fitness: {calculate_fitness(best_solution)}")
```

```
Generation 1: Best fitness = 9, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 0]
Generation 2: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 3: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 4: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 5: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 6: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 7: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 8: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 9: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 10: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 11: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 12: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 13: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 14: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 15: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 16: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 17: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 18: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 19: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 20: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 21: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 22: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 23: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 24: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 25: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

SUB: - SOFT COMPUTING PRACTICAL

The image displays two screenshots of a Google Colab notebook titled 'Untitled6.ipynb'. The notebook is running a genetic algorithm, and the output shows the results for each generation. The first screenshot shows generations 23 to 48, and the second screenshot shows generations 29 to 50. In both, the 'Best fitness' remains constant at 10, and the 'Best individual' is consistently [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]. The final output in the second screenshot states: 'Final best solution found: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1] with fitness: 10'.

```
Generation 23: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 24: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 25: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 26: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 27: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 28: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 29: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 30: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 31: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 32: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 33: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 34: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 35: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 36: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 37: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 38: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 39: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 40: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 41: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 42: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 43: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 44: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 45: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 46: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 47: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 48: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]

Generation 29: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 30: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 31: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 32: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 33: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 34: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 35: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 36: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 37: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 38: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 39: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 40: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 41: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 42: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 43: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 44: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 45: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 46: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 47: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 48: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 49: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Generation 50: Best fitness = 10, Best individual = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]

Final best solution found: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1] with fitness: 10
```

10(B):-Create two classes City and Fitness using Genetic algorithm.

CODE:-

```
import random
import numpy
as np
```

```
# Class to represent a city with x, y coordinates
class City:
```

```
def init_(self, x, y): self.x =
    x self.y = y
```

```
def distance(self, city): x_dis =
    abs(self.x - city.x) y_dis
    = abs(self.y - city.y) distance = np.sqrt((x_dis ** 2)
    + (y_dis ** 2)) return distance
```

```
def _repr_(self):
    return f"({self.x}, {self.y})"
```

Class to calculate the fitness of a route (chromosome) class Fitness:

```
def init_(self, route):
    self.route = route
    self.distance = 0
    self.fitness = 0.0
```

```
def route_distance(self): if
    self.distance == 0:
    path_distance = 0 for i in
    range(0, len(self.route)):
    from_city = self.route[i] to_city =
    None if i + 1 < len(self.route):
        to_city = self.route[i + 1]
    else:
        to_city = self.route[0] # Return to the starting city
    path_distance += from_city.distance(to_city)
    self.distance = path_distance
    return self.distance
```

```
def route_fitness(self):
    if self.fitness == 0.0:
        self.fitness = 1 / float(self.route_distance()) return
    self.fitness
```

--- Genetic Algorithm Simulation (Simplified) ---

1. Create cities city_list

```
= [
    City(60, 200), City(180, 200), City(80, 180), City(140, 180), City(20,
    160), City(100, 160), City(200, 160), City(40, 140),
    City(120, 140), City(180, 120), City(60, 100), City(120, 80),
    City(180, 60), City(20, 40), City(100, 40), City(200, 40)
]
```

2. Generate a random initial population (routes)

```
population_size = 5 initial_population = [] for _ in
    range(population_size): route =
```

```
random.sample(city_list, len(city_list)) initial_population.append(route)
```

```
print("--- Initial Population (Routes) ---") for i,
route in enumerate(initial_population):
    print(f"Route {i+1}: {route}")
```

```
# 3. Evaluate fitness of initial population print("\n---
Fitness Evaluation ---")
fitness_scores = [] for i, route in
enumerate(initial_population): fitness_obj =
Fitness(route) score =
fitness_obj.route_fitness()
fitness_scores.append((route, score))
print(f"Route {i+1} Fitness: {score:.4f}")
```

```
# 4. Select the best route from the initial population
best_route = None
highest_fitness = -1 for route,
score in fitness_scores: if score >
highest_fitness: highest_fitness =
score best_route = route
```

```
print(f"\n--- Best Route in Initial Population ---")
print(f"Route: {best_route}")
print(f"Fitness: {highest_fitness:.4f}")
print(f"Distance: {1/highest_fitness:.2f}")
```

SUB: - SOFT COMPUTING PRACTICAL

OUTPUT:-

```

--- Initial Population (Routes) ---
Route 1: [(40, 140), (180, 60), (200, 160), (60, 100), (80, 180), (100, 160), (180, 200), (180, 120), (100, 40), (140, 180), (200, 40), (120, 80), (20, 40), (170, 140), (20, 160)]
Route 2: [(120, 140), (180, 120), (20, 40), (60, 100), (20, 160), (100, 40), (40, 140), (60, 200), (120, 80), (180, 200), (200, 40), (140, 180), (100, 160), (80, 180), (200, 160)]
Route 3: [(180, 120), (200, 160), (40, 140), (100, 40), (120, 140), (20, 160), (180, 60), (120, 80), (60, 100), (140, 180), (60, 200), (200, 40), (180, 200), (20, 40), (100, 160)]
Route 4: [(140, 180), (20, 160), (120, 80), (40, 140), (140, 140), (200, 160), (180, 60), (100, 40), (100, 160), (80, 180), (60, 200), (200, 40), (180, 200), (60, 100), (180, 120), (120, 140)]
Route 5: [(120, 80), (20, 160), (60, 200), (200, 40), (200, 160), (100, 40), (120, 140), (180, 200), (180, 120), (140, 180), (60, 100), (20, 40), (80, 180), (100, 160), (180, 60)]

--- Fitness Evaluation ---
Route 1 Fitness: 0.0006
Route 2 Fitness: 0.0006
Route 3 Fitness: 0.0005
Route 4 Fitness: 0.0005
Route 5 Fitness: 0.0006

--- Best Route in Initial Population ---
Route: [(40, 140), (180, 60), (200, 160), (60, 100), (80, 180), (100, 160), (180, 200), (180, 120), (100, 40), (140, 180), (200, 40), (120, 80), (20, 40), (120, 140), (20, 160)]
Fitness: 0.0006
Distance: 1667.15

```